

《数据结构》

1. 数据

数据是对客观事物的符号表示，在计算机科学中是指所有能输入到计算机中并被计算机程序处理的符号的总称。

2. 数据元素

数据元素是数据的基本单位，数据项是数据的不可分割的最小单位。

3. 数据对象

数据对象是性质相同的数据元素的集合。

4. 数据结构

数据的逻辑结构是相互之间存在一种或多种特定关系的数据元素的集合，形式定义为一个二元组 $\text{Data_Structure}=(D, S)$

其中，D 是数据元素的有限集；S 是 D 上关系的有限集。

(1) 集合：数据元素之间除了“同属于一个集合”的关系外，别无其他关系。

(2) 线性结构：数据元素之间存在一个一对一的关系，有且仅有一个开始结点和终端结点，除开始结点外，每个结点有且仅有一个前趋结点，除终端结点外，每个结点有且仅有一个后继结点。

(3) 树型结构：数据元素之间存在一个对多个的关系，有一个开始结点和多个终端结点，除开始结点外，每个结点有且仅有一个前趋结点，除终端结点外，每个结点可能有多个后继结点。

(4) 网状结构或图状结构：数据元素之间存在多个对多个的关系，每个结点可能有多个前趋结点和多个后继结点。

树型结构和网状结构又称为非线性结构。

顺序存储、链接存储、索引存储、散列存储。

1.2 算法及其分析

算法是指令的有限运算序列。

算法具有以下 5 个重要特性：

1. 有穷性；2. 确定性；3. 可行性；4. 输入；5. 输出

1.3 算法的性能分析与度量

1. 算法的性能标准

(1) 正确性；(2) 可读性；(3) 健壮性；(4) 高效率与低存储量

2. 算法效率的度量

时间复杂度 $T(n)=O(f(n))$

空间复杂度 $S(n)=O(f(n))$

算法+数据结构=程序

第二章 线性表

2.1 线性表的顺序存储

1. 线性表的顺序存储特点

逻辑上相邻的元素，物理位置也相邻。

静态有序表。

线性表的顺序存储结构是一种随机存取的存储结构。

表 2-1 插入、删除、查找、合并算法的时间复杂度

操作	元素移动 (或比较) 最少	元素移动 (或比较) 最多	元素移动 (或比较) 平均
插入	0	n	n/2
删除	0	n-1	(n-1)/2
查找	1	n	(n+1)/2
有序表的合并	n_1+n_2	n_1+n_2	n_1+n_2

2.2 线性表的动态链式存储

特点：逻辑上相邻的元素，物理位置可以不相邻，表中元素只能顺序访问，插入、删除等操作只需修改指针而不需移动元素。

线性表的链式存储结构是一种顺序存取的存储结构。

2.3 线性表的静态链式存储

特点：用一维数组描述线性链表。

2.4 其他形式的链表

1. 循环线性链表

特点：最后一个结点的指针域指向头结点，从任意结点出发都可以找到表中的其他结点。

2. 双向链表

特点：双向链表克服了在单链表中求前趋需要遍历链表而导致效率较低的缺点。从任意结点出发都可以找到表中的其他结点。

第3章 栈和队列

栈和队列是操作受限制的线性表。

3.1 栈及其应用

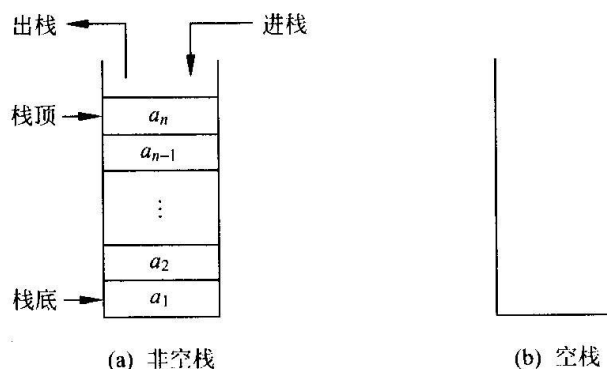
栈(stack)是限定只在表尾(栈顶)进行插入或删除操作的线性表，是后进先出(LIFO)的线性表。

栈结构通常采用的两种存储结构是顺序存储结构和链表存储结构。

图3.1 栈结构的示意图

(A, B, C, D)

(C, A, B, D)



3.2 队列及其应用

队列(Queue)是限定只能在表的一端

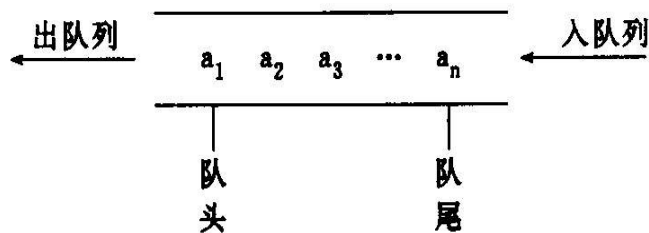
(队尾)进行插入，而在表的另一端(队头)进行删除操作的线性表。和栈相反，队列是一种先进先出(FIFO)的线性表。

图3.2 队列的示意图

链队列：队列的链式存储结构

循环队列：队列的顺序存储结构

1, 2, 3, 4



线性表、栈和队列都是线性结构，可以在线性表的任何位置插入和删除元素；对于栈只能在栈顶插入和删除元素；对于队列只能在队尾插入元素和在队首删除元素。

第4章 串

4.1 有关概念

串(或字符串)，是由零个或多个字符组成的有序序列。

$s = 'a_1 a_2 \dots a_n'$ ($n \geq 0$)

串中字符的数目称为串的**长度**。零个字符的串称为**空串**，它的长度为零。
 串中任意个连续的字符组成的子序列称为该串的**子串**。包含子串的串相应地称为**主串**。
 空串是所有字符串的子串，任何字符串是自身的子串。

4.2 串的存储表示

1. 串的定长顺序存储表示

用一组地址连续的存储单元存储串值的字符序列，每个串变量分配一个固定长度的存储区。

2. 串的变长顺序存储(堆分配存储)表示

用一组地址连续的存储单元存储串值的字符序列，它们的存储空间是在程序执行过程中动态分配而得，对字符串进行操作时不存在溢出问题。

3. 串的块链存储表示

适用于不改变数据结点的操作，如查找(或模式匹配)。

4.3 串的常用运算

串赋值 StrAssign、**串的比较** StrCompare、**求串长** StrLength、**串连接** Strcat 以及 **求子串** SubString 构成串类型的最小操作子集。

如果 s='I AM A STUDENT'，t='GOOD'，则 LENGTH(s) 的结果是 14，

CONCAT(SUBSTR(s, 6, 2), CONCAT(t, SUBSTR(s, 7, 8)))

A GOOD STUDENT

A='BEI' B='JING' C=' ' D='BEIJING'

StrLength(S) 求串长

StrLength(A)=3 StrLength(B)=4

Concat(&T, S1, S2) 联接函数

Concat(&T, A, B) T='BEIJING'

Concat(&T, B, A) T='JINGBEI'

SubString(&Sub, S, pos, len) 求子串

SubString(&Sub, D, 1, 3)='BEI'

SubString(&Sub, D, 4, 0)=''

SubString(&Sub, D, 4, 4)='JING'

Index(S, T, pos) 定位函数

Index(d, b, pos)=4

Index(d, c, pos)=0

如果设 e='I'

则 Index(d, e, pos)=3

Replace(&S, T, V) 置换操作

例如，设 S='BBABBABBA'，T='AB'，V='C'

则 Replace(S, T, V) 的操作结果为 S='BBCBCBA'。

StrInsert(&S, pos, T) 插入操作

例如：StrInsert(&B, 1, A)='BEIJING'

StrDelete(&S, pos, len) 删除操作

例如：StrDelete(&D, 1, 3)='JING'

例如，假设 s='ZHONGGUO'

Concat(SubString(s, 1, 5), ' ', SubString(s, 6, 3))

'ZHONG GUO'

Concat(SubString(s, 1, 2), SubString(s, 7, 2))

'ZHUO'

SubString(d, 1, 3)='NAN'

S='NANJING'

5.1 多维数组

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}$$

m 行 n 列的二维数组

类型特点：

- 1) 只有引用型操作，没有加工型操作；
- 2) 数组是多维的结构，而存储空间是一个一维的结构。

有两种顺序映象的方式：

- 1) 以行序为主序(低下标优先)；
- 2) 以列序为主序(高下标优先)。

$$LOC(a_{ij}) = LOC(a_{11}) + ((i-1)*n + j-1) * L$$

↑
称为基地址或基址。

5.2 特殊矩阵的压缩存储

对称矩阵

$$A = \begin{pmatrix} 3 & 6 & 4 & 7 & 8 \\ 6 & 2 & 8 & 4 & 2 \\ 4 & 8 & 1 & 6 & 9 \\ 7 & 4 & 6 & 0 & 5 \\ 8 & 2 & 9 & 5 & 7 \end{pmatrix}$$

$$k = I * (I - 1) / 2 + J - 1$$

下（上）三角矩阵

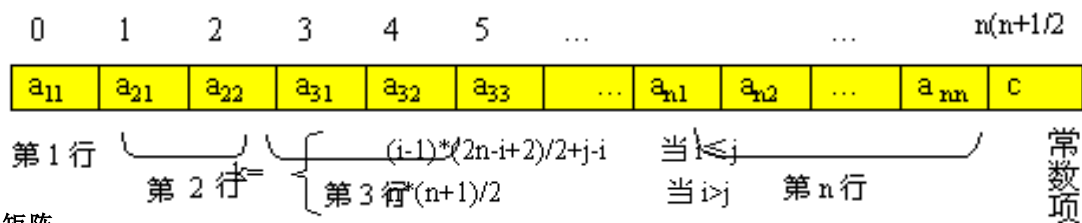
$$\begin{pmatrix} 3 & c & c & c & c \\ 6 & 2 & c & c & c \\ 4 & 8 & 1 & c & c \\ 7 & 4 & 6 & 0 & c \\ 8 & 2 & 9 & 5 & 7 \end{pmatrix}$$

(a) 下三角矩阵

$$\begin{pmatrix} 3 & 4 & 8 & 1 & 0 \\ c & 2 & 9 & 4 & 6 \\ c & c & 1 & 5 & 7 \\ c & c & c & 0 & 8 \\ c & c & c & c & 7 \end{pmatrix}$$

(b) 上三角矩阵

$$k = \begin{cases} i*(i-1)/2+j-1 & \text{当 } i \geq j \\ n*(n+1)/2-1 & \text{当 } i < j \end{cases}$$

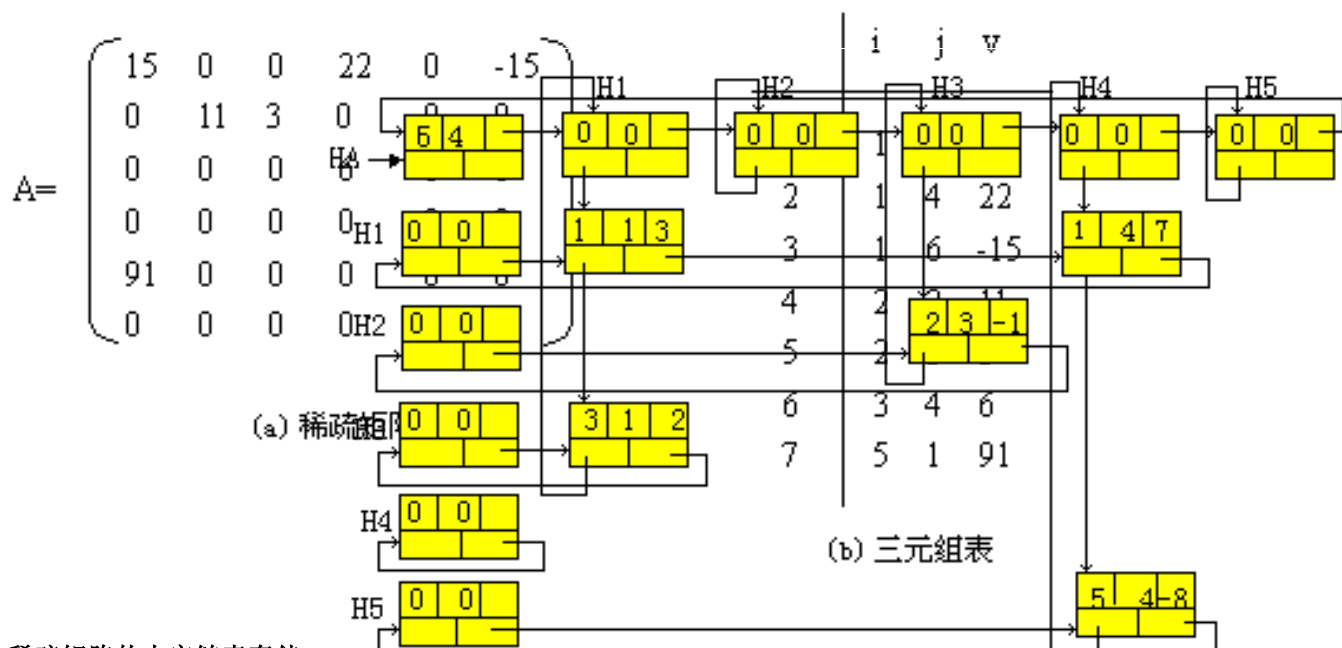


带状矩阵

5.3 稀疏矩阵

稀疏矩阵的三元组表存储

下三角矩阵的压缩存储



稀疏矩阵的十字链表存储

用十字链表表示的稀疏矩阵 A



5.4

A = ()

$$B = (e)$$
$$C = (a, (b, c, d))$$
$$D = (A, B, C)$$
$$E = (a, E)$$
$$F = (\quad)$$

广义表基本运算

Head(1s): 返回广义表 1s 的头部

Tail(1s): 返回广义表的尾部

$$\text{Head (B)} = e \quad \text{Tail (B)} = ()$$

Head (C) = a Tail (C) = ((b, c, d))

Head (D) = A Tail (D) = (B, C)

$$\text{Head}(E) = a \quad \text{Tail}(E) = (E)$$

Head (F) = () Tail (F) = ()

广义表的存储 1. 头尾表示法



2. 孩子兄弟表示法

第六章 树与二叉树

6.1 树的结构特性

树中只有根结点没有前趋，其余结点有且仅有一个前趋，所有结点可有零个或多个后继。

6.2 二叉树及其性质

二叉树：是结点的有限集，该集合或者为空或者是由一个根结点和两棵互不相交的被称为该根的左子树和右子树所组成。

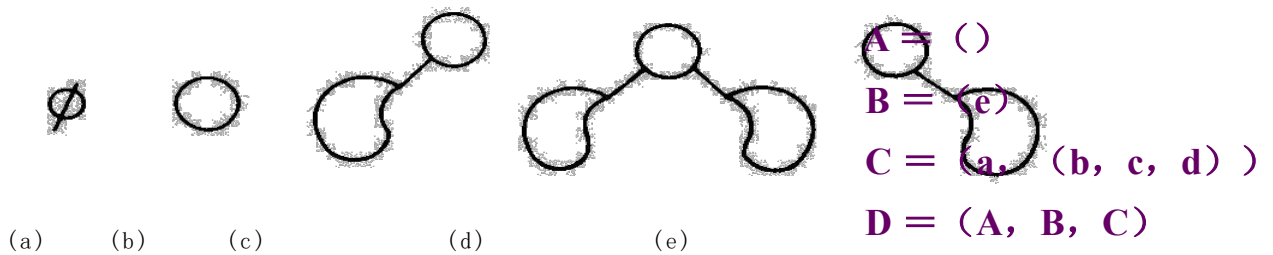
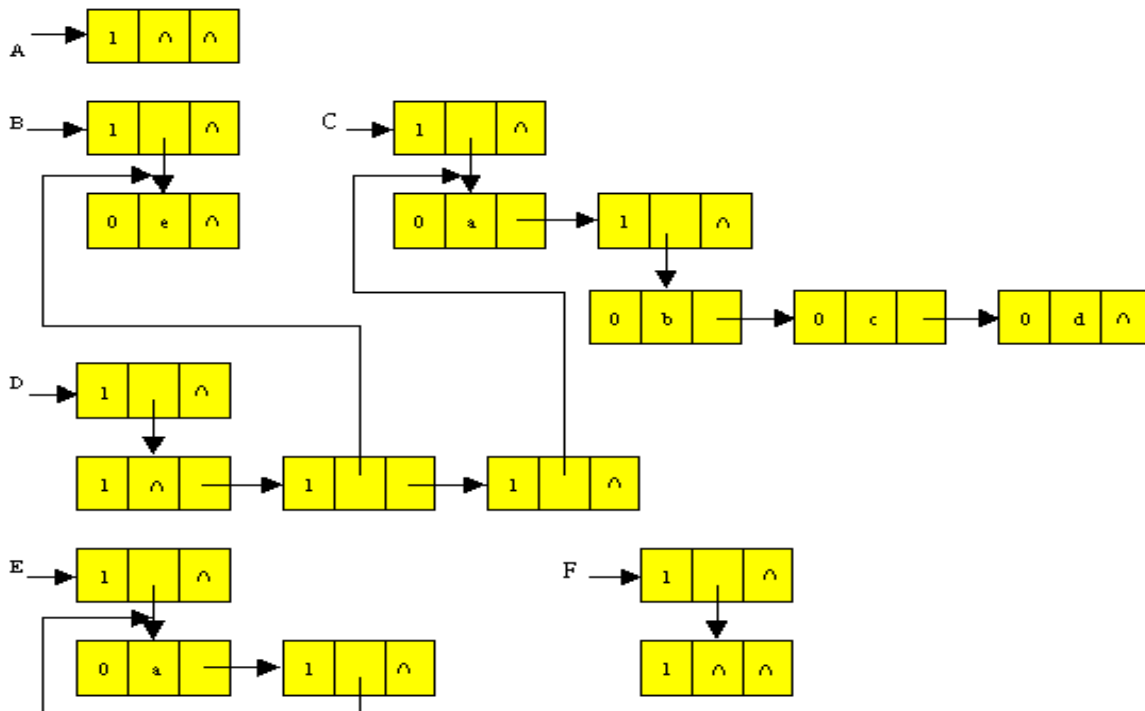


图 5.1 二叉树的五种基本形态

(a) 空二叉树；(b) 仅有根结点的二叉树，(c) 右子树为空的二叉树；

(d) 左、右子树均非空的二叉树；(e) 左子树为空的二叉树。

性质 1 在二叉树的第 i 层上至多有 2^{i-1} 个结点 ($i \geq 1$)。



广义表的孩子兄弟表示法存储结构示例

性质 2 深度为 k 的二叉树至多有 $2^k - 1$ 个结点，($k \geq 1$)。

满二叉树：深度为 k 且结点数为 $2^k - 1$ 的二叉树。

完全二叉树：深度为 k 、有 n 个结点的二叉树，当且仅当每一个结点都与深度为 k 的满二叉树中编号从 1 到 n 的结点一一对应。

性质 3 对任何一棵二叉树 T ，如果其终端结点数为 n_0 ，而度为 2 的结点数为 n_2 ，则 $n_0 = n_2 + 1$ 。
 $n_0 = 6$ $n_2 = 5$ $n_0 = n_2 + 1 = 6$

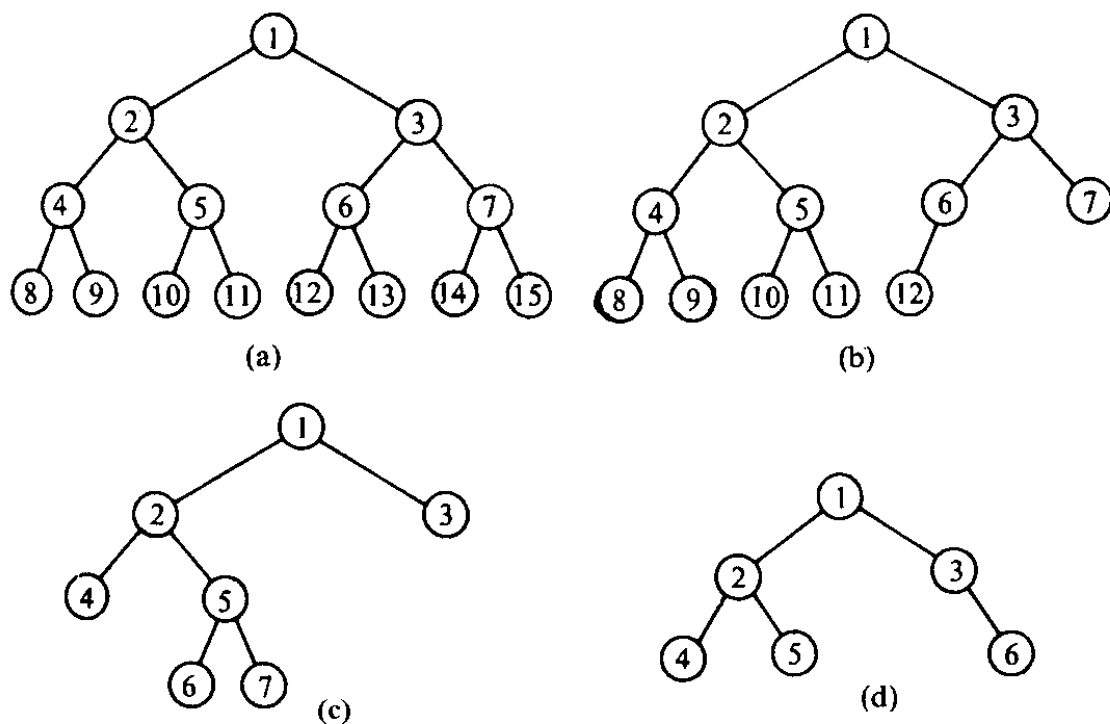


图 5.3 特殊形态的二叉树
 (a) 满二叉树；(b) 完全二叉树；
 (c) 和 (d) 非完全二叉树。

性质 4 具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$

性质 5 如果对一棵有 n 个结点的完全二叉树（其深度为 $\lfloor \log_2 n \rfloor + 1$ ）的结点按层序编号（从第 1 层到第 $\lfloor \log_2 n \rfloor + 1$ 层，每层从左到右），则对任一结点 i ($1 \leq i \leq n$)，有

- (1) 如果 $i = 1$ ，则结点 i 是二叉树的根，无双亲；如果 $i > 1$ ，则其双亲是结点 $\lfloor i / 2 \rfloor$ 。
- (2) 如果 $2i > n$ ，则结点 i 无左孩子；否则其左孩子是结点 $2i$ 。
- (3) 如果 $2i + 1 > n$ ，则结点 i 无右孩子；否则其右孩子是结点 $2i + 1$ 。

(1) n 个结点的二叉树最多有 n 层，最少有 $\lfloor \log_2 n \rfloor + 1$ 层；

(2) 完全二叉树中度为 1 的结点最多 1 个；

(3) 具有 n 个结点的完全二叉树，编号最大的非叶结点是 $\lfloor n/2 \rfloor$ ，编号最小的叶结点是 $\lfloor n/2 \rfloor + 1$ ；

(4) 度为 1 的结点数为 $(n+1) \bmod 2$ ，度为 0 的结点数为 $\lfloor n/2 \rfloor$ ，度为 2 的结点数为 $\lfloor n/2 \rfloor - 1$ 。

6.3 二叉树的存储结构

1. 顺序存储结构

按照完全二叉树的结构对二叉树的结点进行编号。

2. 链式存储结构

有二叉链表、三叉链表、双亲链表、线索链表。

(1) 二叉链表：二叉树的结点由一个数据元素和分别指向它的左、右子树的两个分支构成。

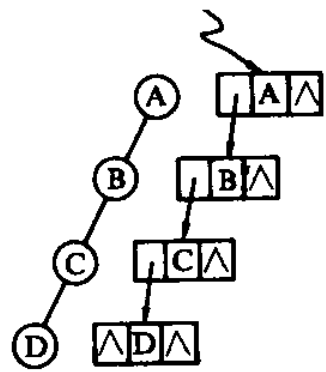
lchild	data	rchild
--------	------	--------

含有两个指针域的结点结构

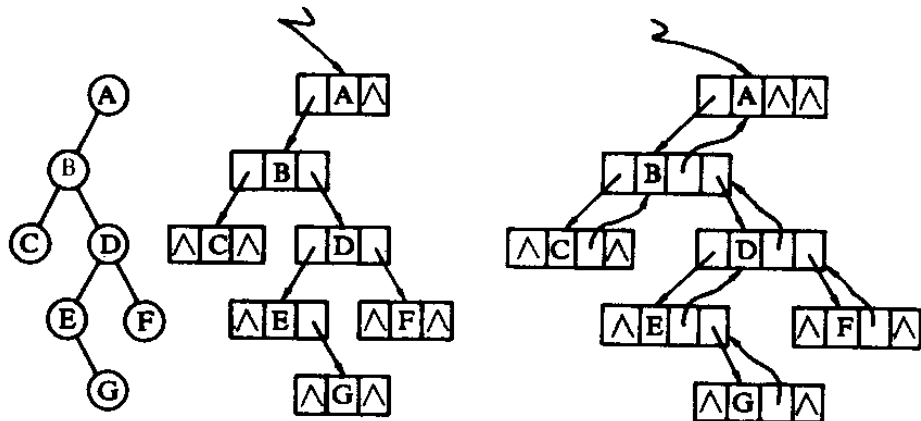
(2) 三叉链表：

lchild	data	parent	rchild
--------	------	--------	--------

含有三个指针域的结点结构



(a)



(b)

(c)

图 5.9 链表存储结构

(a) 单支树的二叉链表；(b) 二叉链表；(c) 三叉链表。

(3) 线索链表

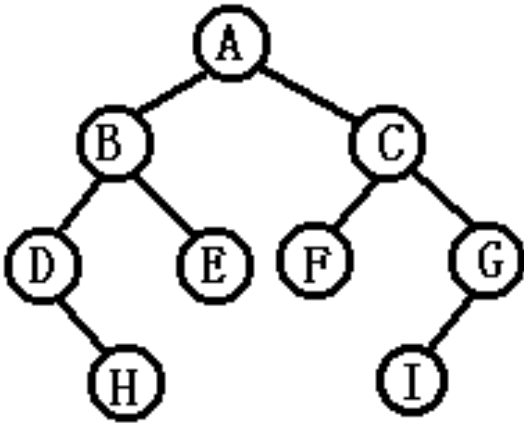
lchild	ltag	data	rtag	rchild
--------	------	------	------	--------

其中： ltag= 0 } lchild 域指示结点的左孩子
 1 } lchild 域指示结点的前驱
 rtag= 0 } rchild 域指示结点的右孩子
 1 } rchild 域指示结点的后继

6.4 遍历二叉树

前序遍历、中序(对称序)遍历、后序遍历、层次(按宽度方向)遍历。

先序遍历为：ABDHECFG I
中序遍历为：DHBEAFCIG
后序遍历为：HDEBFIGCA



6. 5 线索二叉树

线索二叉树：加上线索的二叉树。
线索化：对二叉树以某种次序遍历使其变为线索二叉树的过程。

5. 6 树的存储结构

双亲、孩子链表、树的二叉链表(孩子-兄弟)存储表示法。
1. 双亲表示法
以一组连续空间存储树的结点，同时每个结点中附设一个指示器指示其双亲结点在链表中的位置

data	parent
------	--------

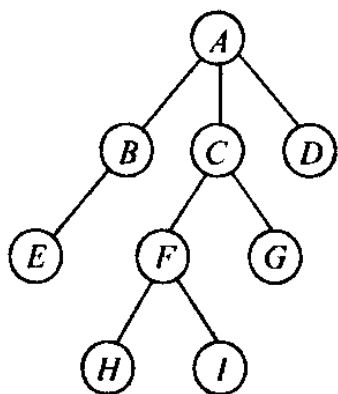
2. 孩子链表表示法
这种存储结构便于那些涉及孩子的操作，但不适合于求结点的双亲。
3. 孩子-兄弟表示法
又称二叉树表示法，或二叉链表表示法。即以二叉链表作树的存储结构(对照树与二叉树的转换) firstchild 域, nextsibling 域。

5. 7 二叉树与树、森林之间的转换

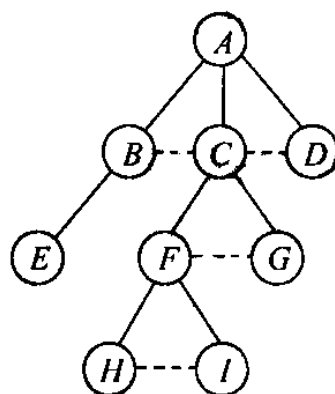
1. 树转换为二叉树的步骤
- (1)加线：在各兄弟结点之间加一连线；
 - (2)抹线：对任何结点，除了其最左边的孩子之外，抹掉该结点原先与其余孩子之间的连线；
 - (3)旋转：将添加的水平连线和原有的连线，以树的根结点为轴心，按顺时针方向粗略地旋转 45°。
- ①此二叉树的根结点只有左子树，而没有右子树；

②转换生成的二叉树中各结点的左孩子是它在原来树中的最左边的孩子，右孩子是它在原来树中的下一个兄弟。

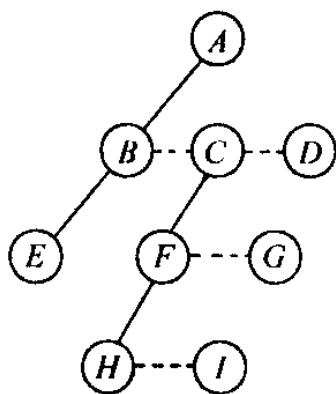
树转换为二叉树的操作过程



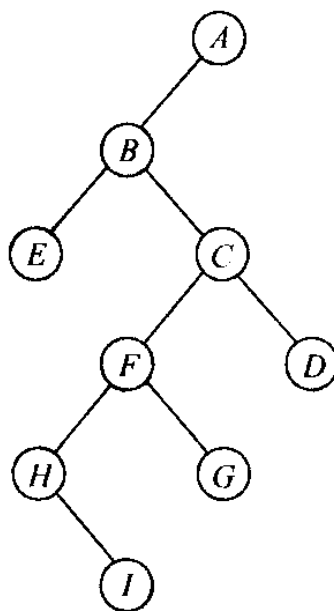
(a) 树



(b) 加线后



(c) 抹线后



(d) 旋转后

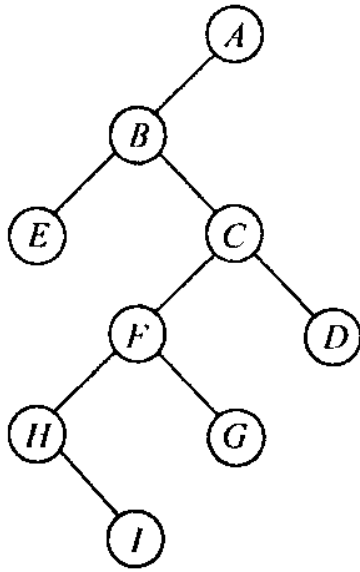
2. 二叉树还原为树的步骤

(1) 加线：如果 p 结点是双亲结点的左孩子，则将 p 结点的右孩子，右孩子的右孩子，……，沿着右分支搜索到的所有右孩子，都分别与 P 结点的双亲用线连起来；

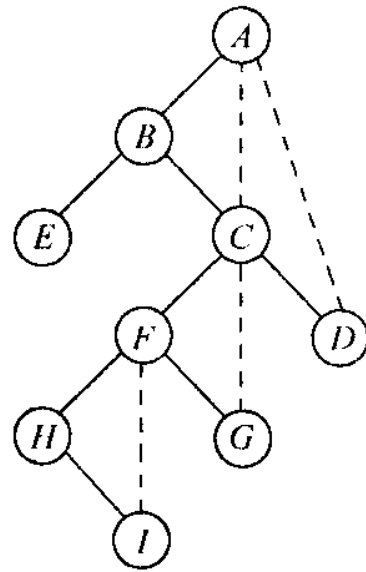
(2) 抹线：抹掉原二叉树中所有双亲结点与右孩子的连线；

(3) 调整：将结点按层次排列，形成树的结构。

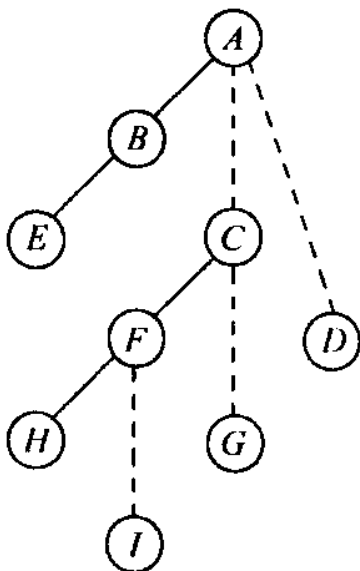
二叉树转换为树的操作过程



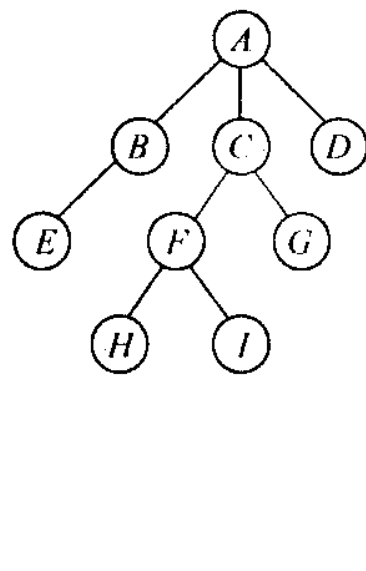
(a) 二叉树



(b) 加线后



(c) 抹线后



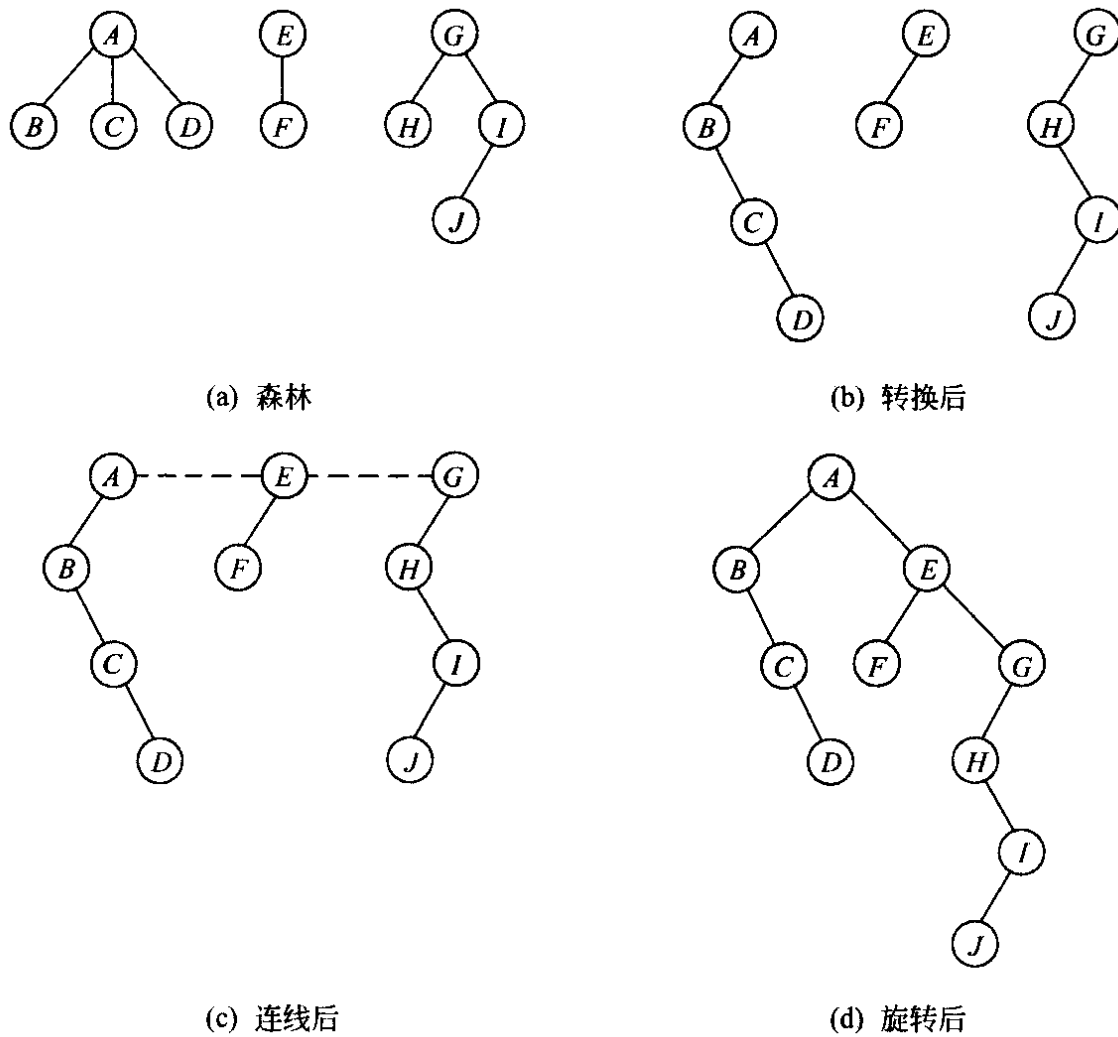
(d) 调整后

3. 森林转换为二叉树的步骤

- (1) 转换：将森林中的每一棵树转换成二叉树；
- (2) 连线：将各棵转换后的二叉树的根结点相连；

(3) 旋转：将添加的水平连线和原有的连线，以第一棵树的根结点为轴心，按顺时针方向粗略地旋转 45° 。

森林转换为二叉树的操作过程

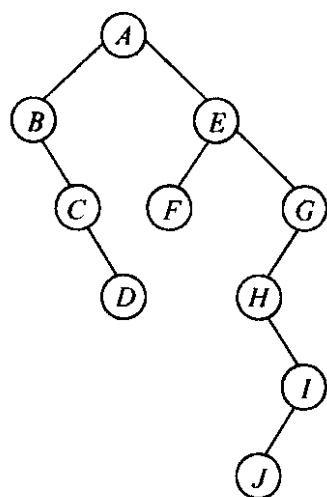


4. 二叉树还原为森林的步骤

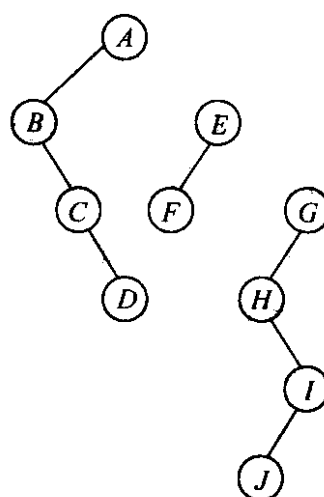
(1) 抹线：抹掉二叉树根结点右链上的所有结点上的连线，分成若干个以右链上的结点为根结点的子二叉树；

(2) 转换：将分好的子二叉树转换成树；

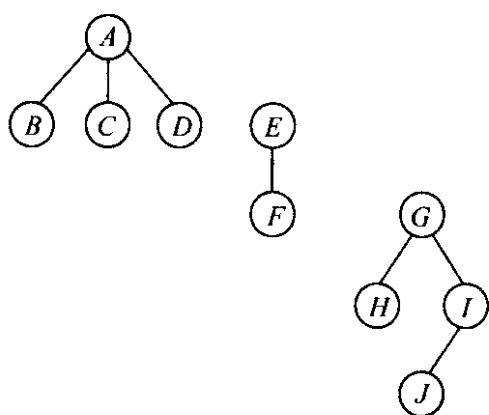
(3) 调整：将转换好的树的根结点排列成一排。



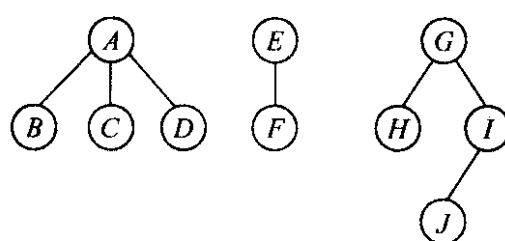
(a) 二叉树



(b) 抹线后



(c) 转换后



(d) 调整后

二叉树还原为森林的操作过程

5. 树和森林的遍历

树有 3 种遍历方法：

(1) 先根遍历

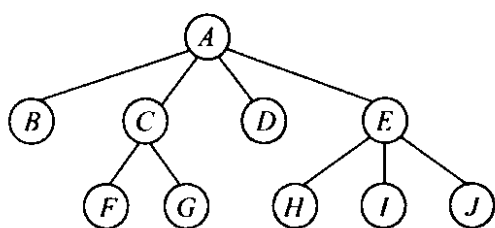
若树非空，则先访问树的根结点，然后依次先根遍历根的每棵子树。树的先根遍历与转换后的二叉树的先序遍历次序一致。

(2) 后根遍历

若树非空，则先依次后根遍历根的每棵子树，然后访问树的根结点。树的后根遍历与转换后的二叉树的中序遍历次序一致。

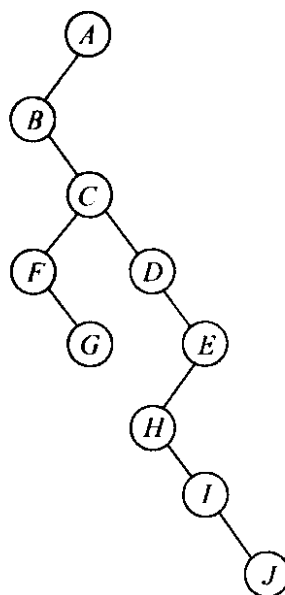
(3) 层次遍历

若树非空，则从树的根结点起按层从左到右依次访问各结点。



(a) 树

树的先根遍历：*ABC FGDEHIJ*
 树的后根遍历：*BFGCDHIJEA*
 树的层次遍历：*ABCDEF GHIJ*



(b) 转换后的二叉树

二叉树的先序遍历：*ABC FGDEHIJ*
 二叉树的中序遍历：*BFGCDHIJEA*

树的先根、后根和层次遍历

(1) 先序遍历

若森林非空，则可以按照下述规则遍历：

- ① 访问森林中第一棵树的根结点；
- ② 先序遍历第一棵树中根结点的子树森林；
- ③ 先序遍历除去第一棵树之后剩余的树构成的森林。

森林的先序遍历与转换后的二叉树的先序遍历次序一致。

(2) 中序遍历

若森林非空，则可以按照下述规则遍历：

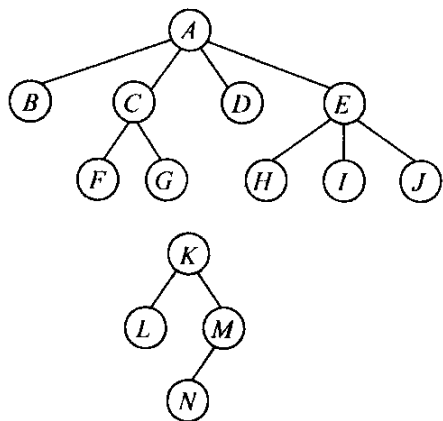
- ① 中序遍历第一棵树中根结点的子树森林；
- ② 访问森林中第一棵树的根结点；
- ③ 中序遍历除去第一棵树之后剩余的树构成的森林。

森林的中序遍历与转换后的二叉树的中序遍历次序一致。

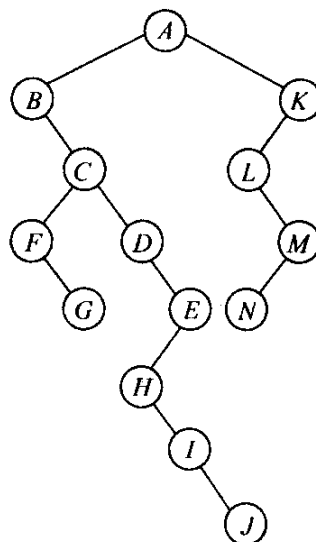
(3) 层次遍历

若森林非空，则可以按照下述规则遍历：

- ① 对第一棵树从根结点起按层从左到右依次访问结点；
- ② 按层访问森林中除第一棵树外剩余的树构成的森林。



(a) 森林



(b) 转换后的二叉树

森林的先序遍历: $ABCDFGDEHIJKLMN$ 二叉树的先序遍历: $ABCDFGDEHIJKLMN$
 森林的中序遍历: $BFGCDHIJEALNMK$ 二叉树的中序遍历: $BFGCDHIJEALNMK$
 森林的层次遍历: $ABCDEFGHIJKLMN$

森林的先根、后根和层次遍历

先序序列、中序序列和层次序列

5. 8 赫夫曼树及其应用

赫夫曼(Huffman)树, 又称**最优二叉树**, 它是 n 个带权叶子结点构成的所有二叉树中, 带权路径长度最短的二叉树。

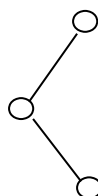
一. 基本概念

1. **路径**: 从树中一个结点到另一个结点之间的分支构成两个结点之间的路径。

A

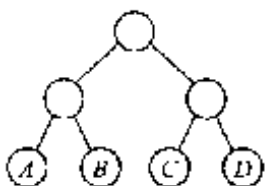
B

C

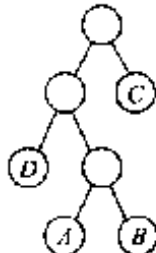


2. 路径长度

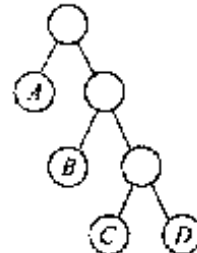
路径上的分支数目称为路径长度。



(a)



(b)



(c)

3. 树的路径长度

从树根到每一个结点的路径长度之和称为树的路径长度。

(a) $2+2+2+2=8$

(b) $3+3+1+2=9$

(c) $1+2+3+3=9$

4. 结点的带权路径长度

从该结点到树根之间的路径长度与结点上权的乘积。

5. 树的带权路径长度

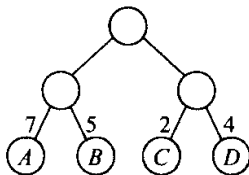
树中所有叶子结点的带权路径长度之和。

$$WPL = \sum_{k=1}^n w_k l_k$$

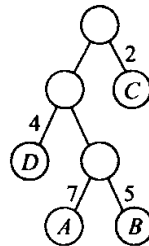
6. 赫夫曼树

带权路径长度 WPL 最小的二叉树称为最优二叉树或赫夫曼树。

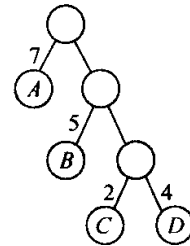
具有不同带权路径长度的二叉树



(a) 带权路径长度为
 $WPL = 7 \times 2 + 5 \times 2 + 2 \times 2 + 4 \times 2 = 36$



(b) 带权路径长度为
 $WPL = 7 \times 3 + 5 \times 3 + 2 \times 1 + 4 \times 2 = 46$



(c) 带权路径长度为
 $WPL = 7 \times 1 + 5 \times 2 + 2 \times 3 + 4 \times 3 = 35$

二. 赫夫曼树的构造

(1) 根据给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ 构成 n 棵二叉树的集合 $F = \{T_1, T_2, \dots, T_n\}$, 其中每棵二叉树 T_i 中只有一个带权 w_i 的根结点 (x_i), 其左子树和右子树均为空;

(2) 在 F 中选取两棵根结点的权值最小的二叉树作为左右子树构造一棵新的二叉树, 且置新的二叉树的根结点的权值为其左、右子树上根结点的权值之和;

(3) 在 F 中删除上面选中的那两棵根结点权值最小的二叉树, 同时将新得到的二叉树加入 F 中;

(4) 重复步骤 (2) 和 (3), 直到 F 中只含一棵树为止。

赫夫曼算法说明

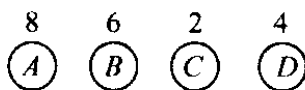
(1) 在选取两棵根结点权值最小的二叉树时, 出现权值相同的情况时, 可以在相同权值的二叉树中选取一棵;

(2) 当两棵根结点的权值最小的二叉树组成新的二叉树的左子树和右子树时, 谁左谁右没有规定

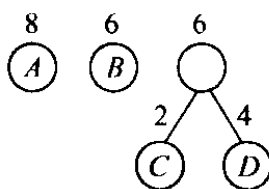
(3) 在赫夫曼树中, 权值越大的叶子结点离根越近;

(4) 在赫夫曼树中没有度为 1 的结点, 根据二叉树性质 $n_0 = n_2 + 1$, 可以推得 n 个叶子结点的赫夫曼树共有 $2n - 1$ 个结点。

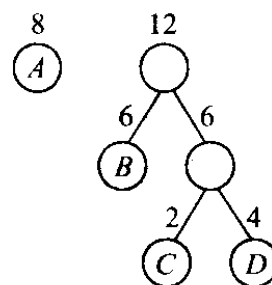
$\{8, 6, 2, 4\}$



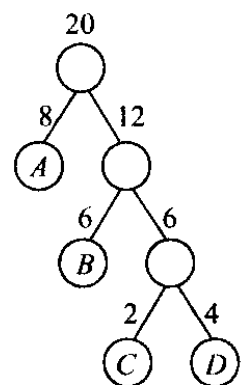
(a) 第一步



(b) 第二步



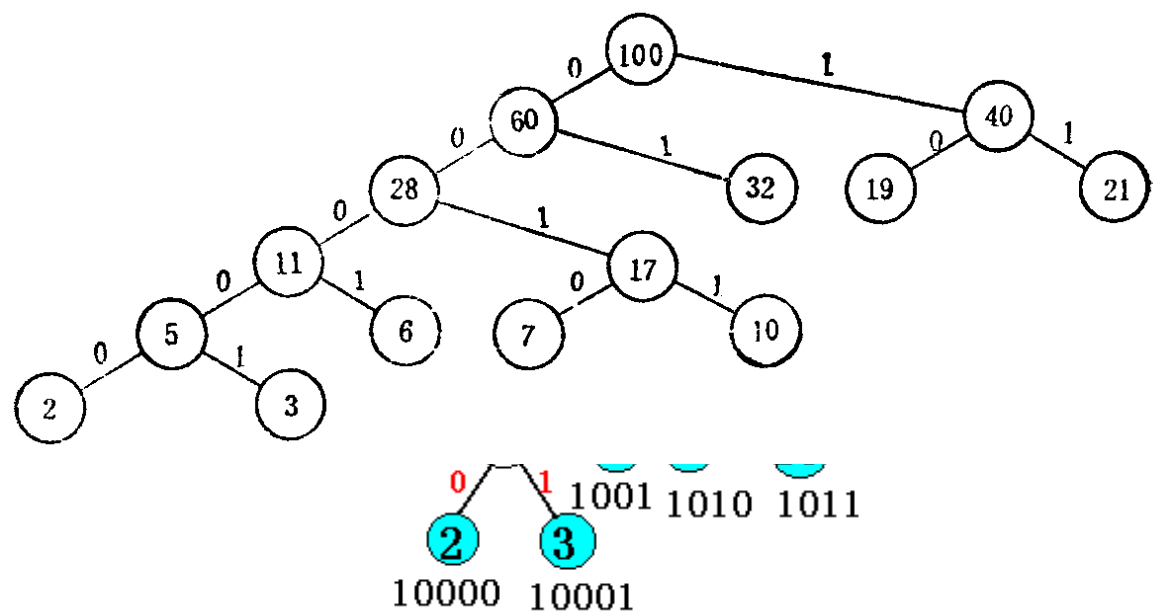
(c) 第三步



(d) 第四步

三. 赫夫曼编码

7, 19, 2, 6, 32, 3, 21, 10



$WPL_{HF}=7\times4+19\times2+2\times5+6\times4+32\times2+3\times5+21\times2+10\times4=261$

$WPL_{EQ}=(7+19+2+6+32+3+21+10)\times3=300$

频 数	7	19	2	6	32	3	21	10
哈夫曼编码 1	1010	00	10000	1001	11	10001	01	1011
哈夫曼编码 2	0010	10	00000	0001	01	00001	11	0111
等长编码	000	001	010	011	100	101	110	111

第六章 图

6. 1 基本概念

图, 有向图, 无向图, 端点和邻接点, 起点和终点, 度、出度和入度, 完全图, 稠密图和稀疏图, 权和网, 路径、路径长度和简单路径, 回路或环, 子图, 连通图和连通分量, 强连通图和强连通分量, 生成树。

图由两个集合 V 和 E 组成, 记为 $G=(V, E)$, 其中 V 是顶点集合, E 是 V 中顶点偶对 (即边) 的集合。用 n 表示图中顶点数目, e 表示图中边或弧的数目。有 $n(n-1)/2$ 条边的无向图称为完全图; 有 $n(n-1)$ 条边的有向图称为有向完全图。有很少条边或弧 (如 $e<n\log n$) 的图称为稀疏图, 反之称为稠密图。

$V_0, V_1, \dots, V_{n-1}, V_n$

无向图: $(v_i, v_j) \in E$

有向图: $\langle v_i, v_j \rangle \in E$

当 $v_0=v_n$ 时此路径称为一条简单回路。

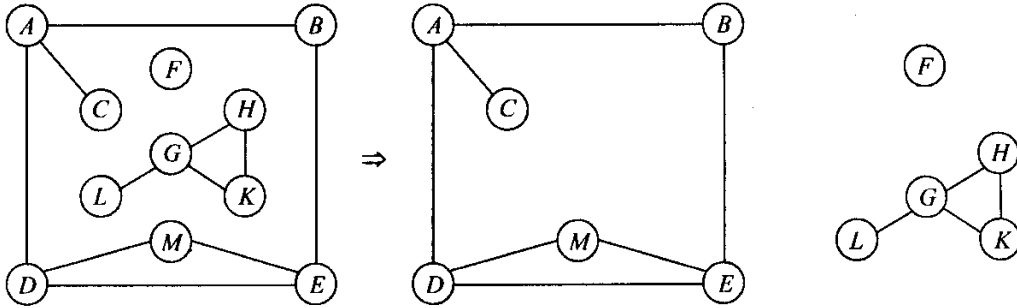
与图的边或弧相关的数叫做权, 带权的图称为网。

连通: 若从顶点 v_i 到顶点 v_j 有路径, 则称 v_i 和 v_j 是连通的。若无向图图中任两点是连通的则称为

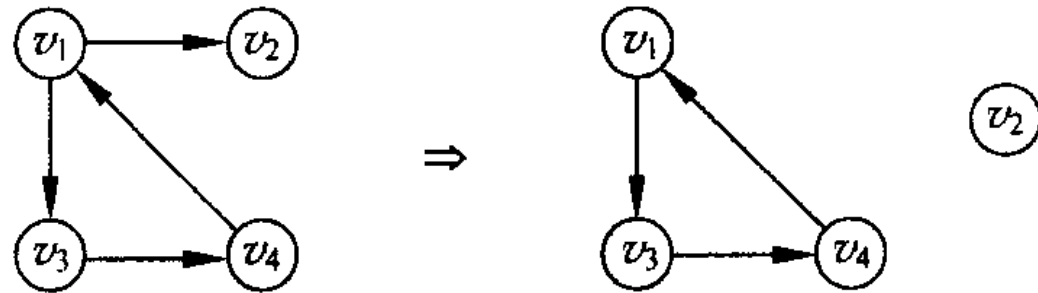
连通图。 n 个顶点的连通图最少有 $n-1$ 条边，最多 $n(n-1)/2$ 条边。

强连通图：有向图中，对任两点 $v_i, v_j (v_i \neq v_j)$ ，存在从 v_i 到 v_j 和 v_j 到 v_i 的路径。 n 个顶点的强连通图最少有 n 条边，最多有 $n(n-1)$ 条边。

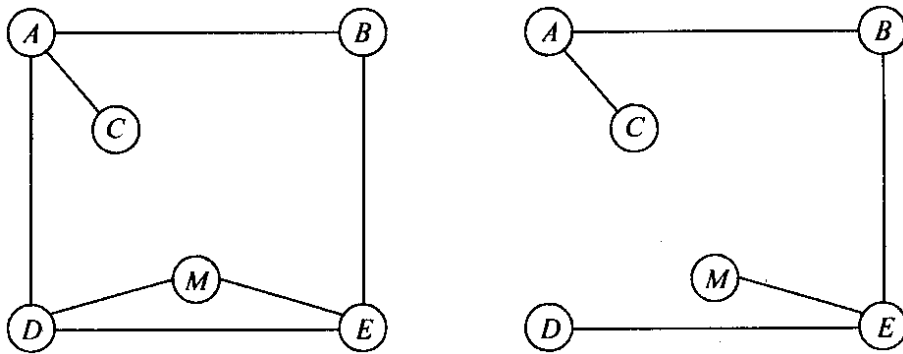
生成树是一个连通图的极小连通子图，它含有图中全部顶点，但只有足以构成一棵二叉树的 $n-1$ 条边，没有回路。有 $n-1$ 条边的图 **不一定** 是生成树。



(a) 无向非连通图 G_3 (b) G_3 的 3 个连通分量
无向非连通图 G_3 和它的 3 个连通分量



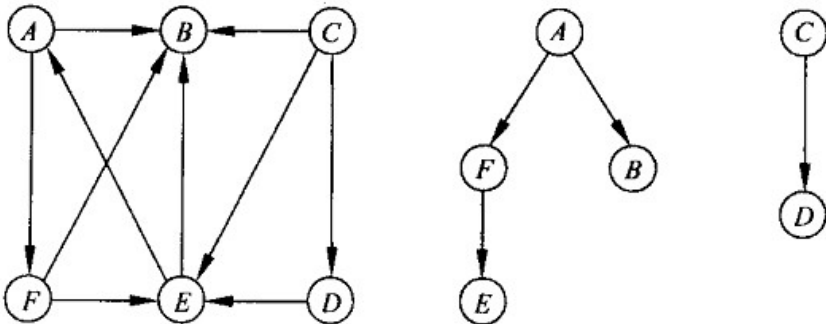
(a) 有向非连通图 G_1 (b) G_2 的两个强连通分量
有向非连通图 G_2 和它的两个强连通分量



(a) 连通图 (b) 连通图的生成树之一
连通图的生成树

生成森林(stanning-forest)：如果一个有向图恰有一个顶点的入度为 0，其余顶点的入度均为 1，则是一棵有向树。一个有向图的生成森林由若干棵有向树组成，含有图中全部顶点，但是只有足以构成若干棵不相交的有向树的弧。

(a) 有向图 G_4 (b) G_4 的生成森林



6.2 图的存储表示

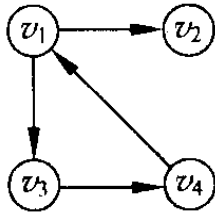
1. 数组表示法(邻接矩阵)

邻接矩阵(adjacency matrix)是图的一种顺序存储结构。

如果图 $G=(V, E)$ 是无权图, 则图 G 的邻接矩阵定义为:

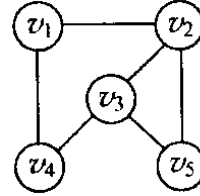
$$A[i][j] = \begin{cases} 1 & (v_i, v_j) \in E \text{ 或 } \langle v_i, v_j \rangle \in E \\ 0 & \text{其他情况} \end{cases}$$

(a) 有向图 G_1 及其邻接矩阵



$$G1.arcs = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

(b) 无向图 G_2 及其邻接矩阵



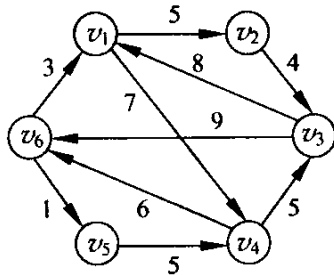
$$G2.arcs = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

图的邻接矩阵

如果图 G 是带权图, 则图 G 的邻接矩阵定义为:

$$A[i][j] = \begin{cases} w_{ij} & (v_i, v_j) \in E \text{ 或 } \langle v_i, v_j \rangle \in E \\ \infty & \text{其他情况} \end{cases}$$

网的邻接矩阵



$$G.arcs = \begin{bmatrix} \infty & 5 & \infty & 7 & \infty & \infty \\ \infty & \infty & 4 & \infty & \infty & \infty \\ 8 & \infty & \infty & \infty & \infty & 9 \\ \infty & \infty & 5 & \infty & \infty & 6 \\ \infty & \infty & \infty & 5 & \infty & \infty \\ 3 & \infty & \infty & \infty & 1 & \infty \end{bmatrix}$$

无向图:

$$TD(v_i) = OD(v_i) + ID(v_i) = \sum_{j=1}^n A[i][j]$$

有向图:

$$TD(v_i) = \sum_{j=1}^n A[i][j] + \sum_{i=1}^n A[i][j]$$

二. 邻接表表示法

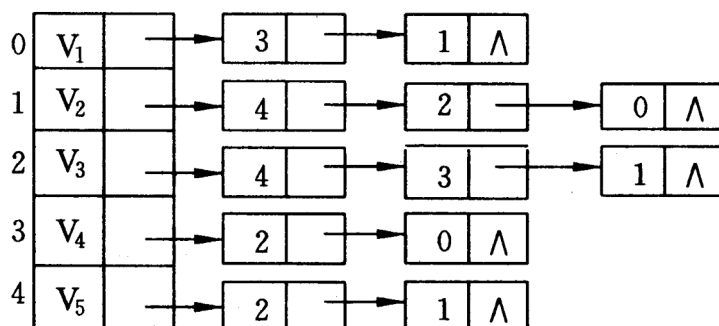
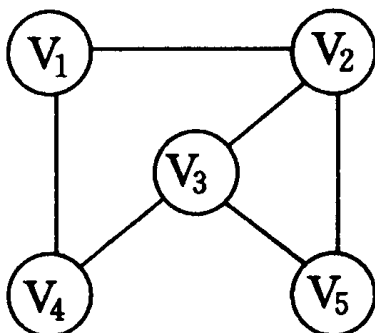
邻接表(Adjacency list)是图的一种链式存储结构, 既适用于无向图又适用于有向图。

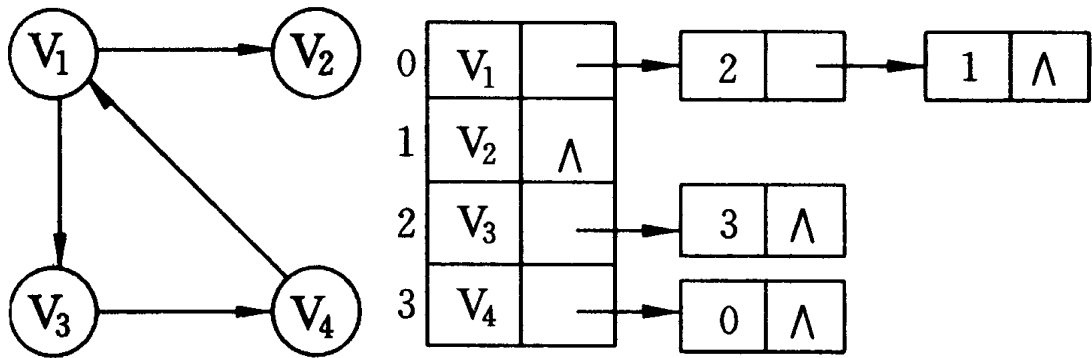
1. 邻接表的定义

(1) 在邻接表中, 对图中的每一个顶点建立一个单链表, 第 i 个单链表中的结点表示依附于顶点 v_i 的边(对有向图是以顶点 v_i 为尾的弧)。

如果无向图中有 n 个顶点、 e 条边, 则它的邻接表需要 n 个头结点和 $2e$ 个表结点。

(a) 无向图 G_2 及邻接表



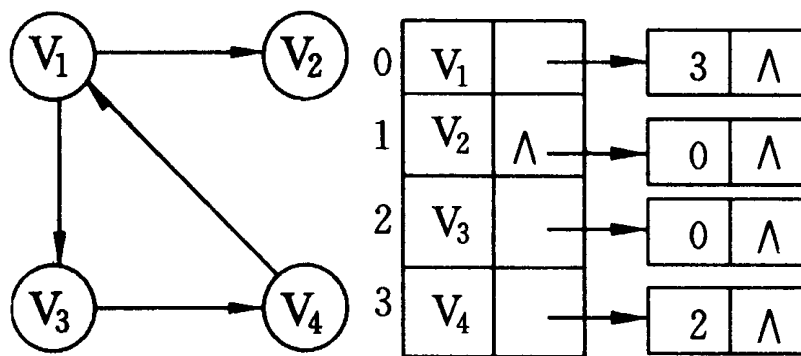


(b) 有向图 G1 及邻接表

图的邻接表表示

(a) 有向图 G1

(b) G1 的逆邻接表



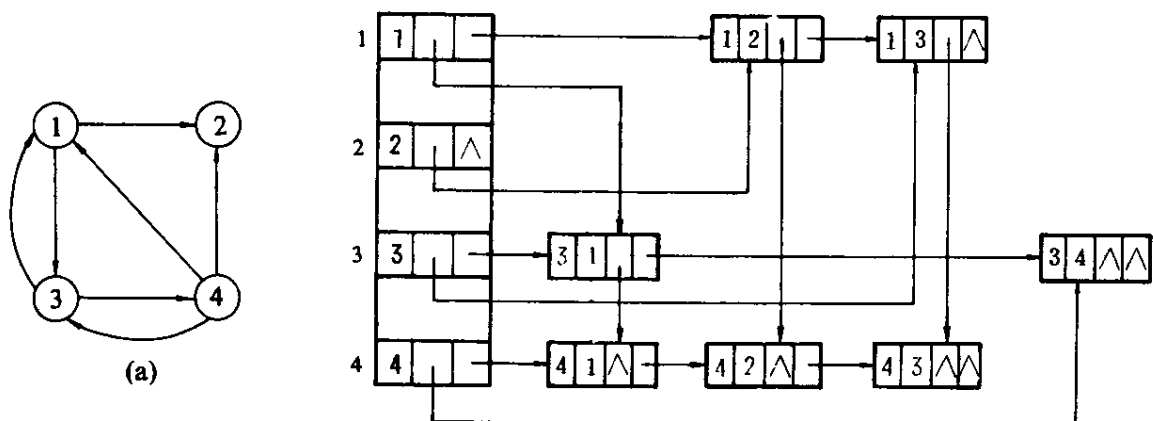
有向图 G1 及逆邻接表

三. 十字链表表示法

十字链表(orthogonal list)是有向图的另一种链式存储结构。是将有向图的邻接表和逆邻接表结合起来得到的一种链表。

在十字链表中，有向图中每一条弧用一个结点表示，每个顶点也用一个结点表示。

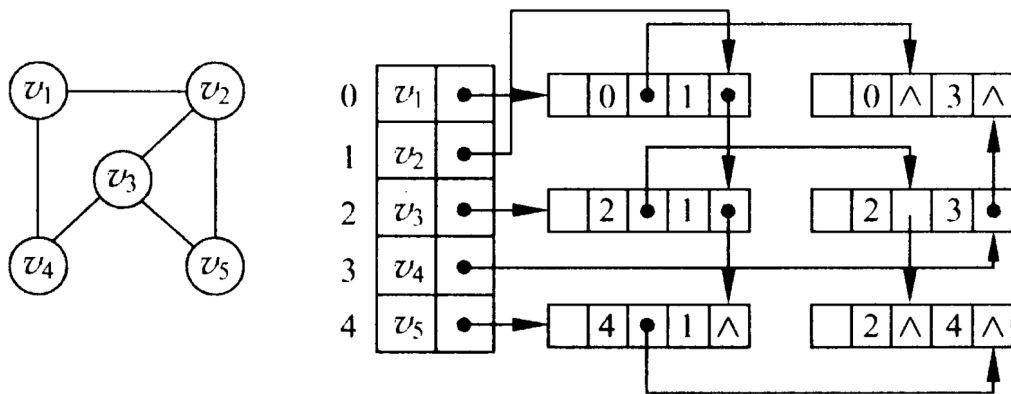
有向图的十字链表



(a)

四. 邻接多重表表示法

邻接多重表(Adjacency Multilist)是无向图的另一种链式存储结构。



无向图 G_2 的邻接多重表

6.3 图的遍历

从图中某一顶点出发访遍图中其余顶点，且使每一个顶点仅被访问一次。这一过程称为图的遍历。

1. 深度优先搜索 DFS (Depth First Search)

从图中某个顶点 v_0 出发，访问此顶点，然后依次从 v_0 的未被访问的邻接点出发深度优先搜索遍历图，直到图中所有和 v_0 有路径相通的顶点都被访问到；若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作始点，重复上述过程，直到图中所有顶点都被访问到为止。

若用邻接矩阵作存储结构，则时间复杂度为 $O(n^2)$

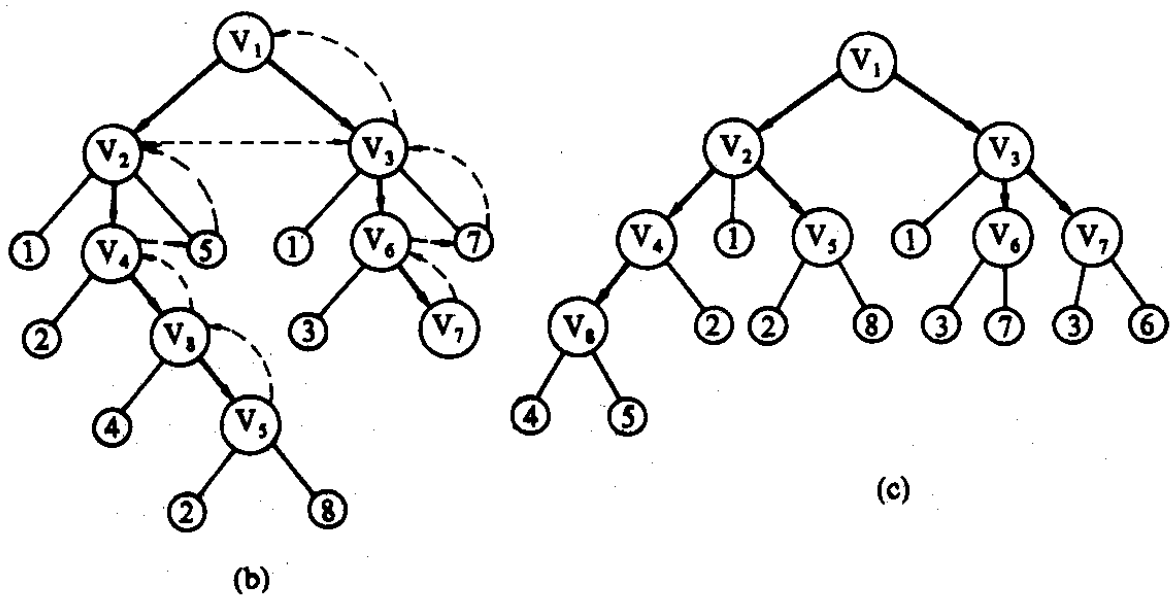
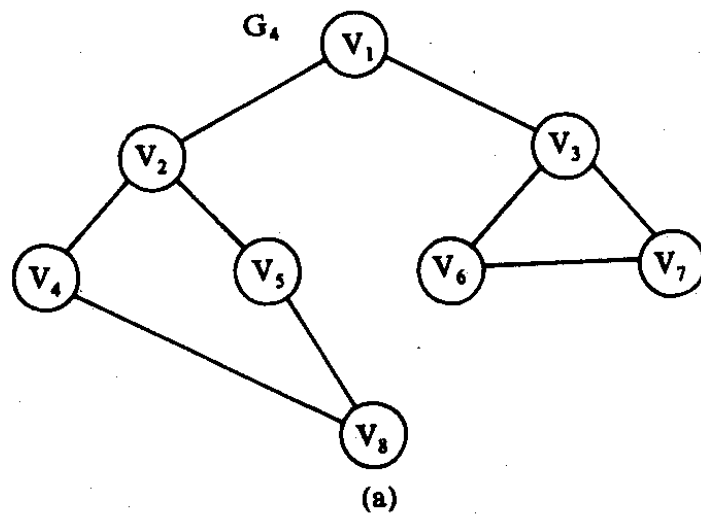
若用邻接表作存储结构，则时间复杂度为 $O(n+e)$

$v_1 \rightarrow v_2 \rightarrow v_4 \rightarrow v_8 \rightarrow v_5 \rightarrow v_3 \rightarrow v_6 \rightarrow v_7$

2. 广度优先搜索 BFS (Breadth First Search)

从图中某顶点 v_0 出发，在访问 v_0 之后依次访问 v_0 的各个未曾访问过的邻接点，并使“先被访问的顶点的邻接点”先于“后被访问的顶点的邻接点”被访问，直至图中所有已被访问的顶点的邻接点都被访问到；然后分别从这些邻接点出发广度优先搜索遍历图，直到图中所有已被访问的顶点的邻接点都被访问到，若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作起始点，重复上述过程直到图中所有顶点都被访问到为止。

$v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_5 \rightarrow v_6 \rightarrow v_7 \rightarrow v_8$



遍历图的过程

(a) 无向图 G_4 ; (b) 深度优先搜索的过程; (c) 广度优先搜索的过程。

6.4 图的连通性

1. (强) 连通分量的确定

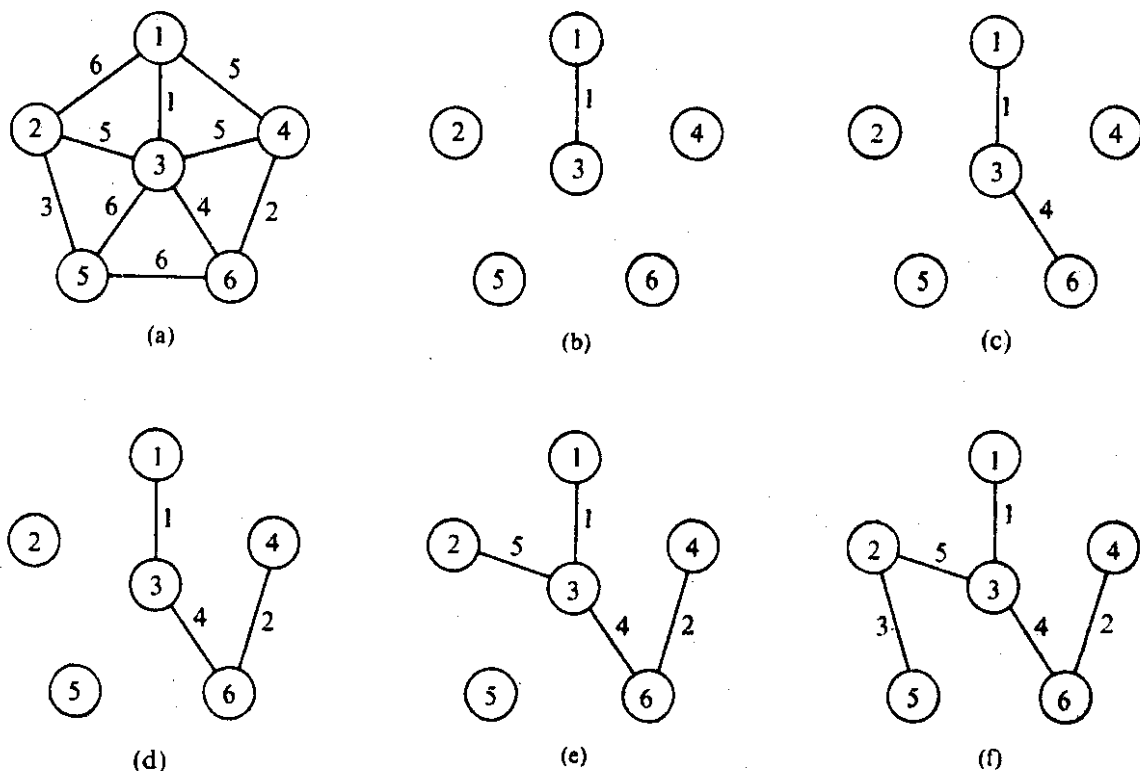
2. 最小(代价)生成树 MST (Minimum cost Spanning Tree)

最小生成树的性质: 假设 $N=(V, \{E\})$ 是一个连通网, U 是顶点集 V 的一个非空子集, 若 (u, v) 是一条具有最小权值(代价)的边, 其中 $u \in U, v \in V-U$, 则必存在一棵包含边 (u, v) 的最小生成树。

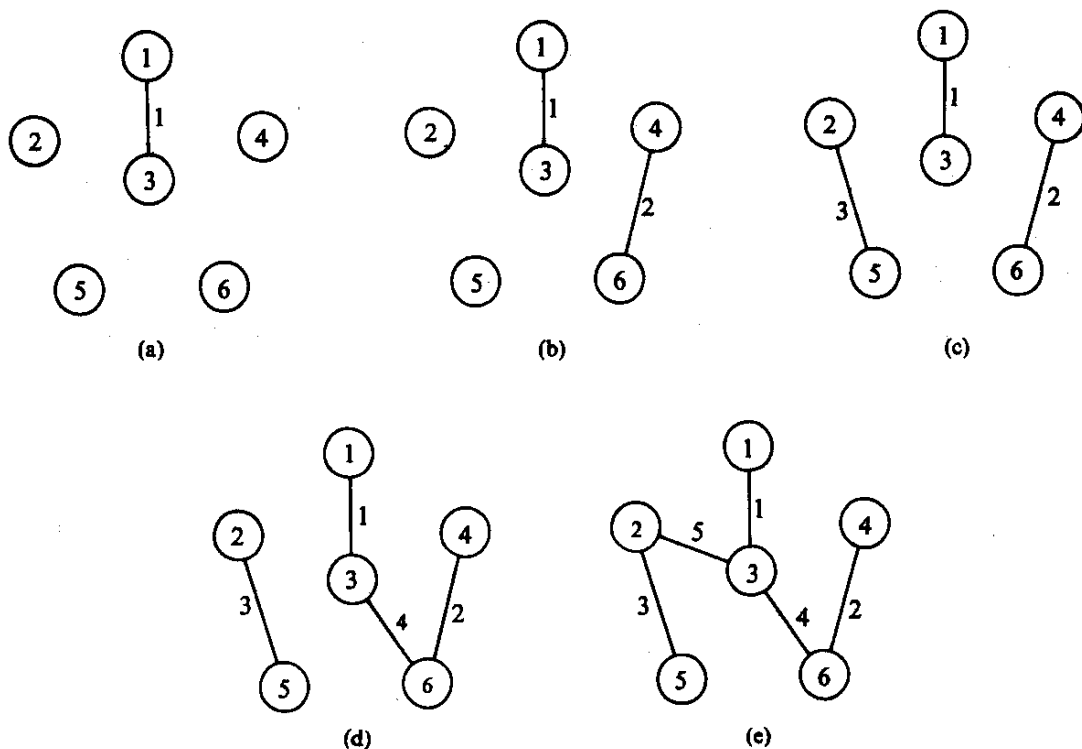
普里姆 (Prim) 算法：假设 $N=(V, \{E\})$ 是连通网， TE 是 N 上最小生成树中边的集合。算法从 $U=\{u_0\} (u_0 \in V), TE=\{\}$ 开始，重复执行下述操作：在所有 $u \in U, v \in V-U$ 的边 $(u, v) \in E$ 中找一条代价最小的边 (u_0, v_0) 并入集合 TE ，同时 v_0 并入 U ，直至 $U=V$ 为止。此时 TE 中必有 $n-1$ 条边，则 $T=(V, \{TE\})$ 为 N 的最小生成树。

图 6.24 普里姆算法构造最小生成树的过程

克鲁斯卡尔 (Kruskal) 算法：假设 $N=(V, \{E\})$ 是连通网，则令最小生成树的初始状态为只有 n 个顶点而无边的非连通图 $T=(V, \{\})$ ，图中每个顶点自成一个连通分量。在 E 中选择代价最小的边，若该



边依附的顶点落在 T 中不同的连通分量上，则将此边加入到 T 中，否则舍去此边而选择下一条代价最小的边。依次类推，直至 T 中所有顶点都在同一连通分量上为止。



克鲁斯卡尔算法构造最小生成树的过程

普里姆算法的时间复杂度为 $O(n^2)$ (与网中边数无关), 适用于求边稠密的网的最小生成树。而克鲁斯卡尔算法的时间复杂度为 $O(e \log 2e)$, 适用于求边稀疏的网的最小生成树。

第七章 查找

7.1 基本概念

查找表是由同一类型的数据元素(或记录)构成的集合。

对查找表经常进行的操作有:

- (1) 查询某个“特定的”数据元素是否在查找表中;
- (2) 检索某个“特定的”数据元素的各种属性;
- (3) 在查找表中插入一个数据元素;
- (4) 从查找表中删去某个数据元素。

关键字(key): 数据元素(或记录)中某个数据项的值, 用它可以标识(识别)一个数据元素(或记录)。若此关键字可以惟一地标识一个记录, 则称此关键字为主关键字; 反之称用以识别若干记录的关键字为次关键字。

查找(或称检索): 根据给定的某个值, 在查找表中确定一个其关键字等于给定值的记录或数据元素。若表中存在这样的一个记录, 则称查找是成功的; 若表中不存在关键字等于给定值的记录, 则称查找不成功。

7.2 静态查找表

1. 线性表的顺序查找

顺序查找的过程就是从表的一端开始逐个进行记录的关键字和给定值的比较, 直到查找成功或查找完整个表(不成功)。

平均查找长度为 $(n+1)/2$

2. 有序表的查找

有序表的查找方法有折半(或二分)查找。

设查找范围为 $[low, high]$, 在 low 与 $high$ 之间找一中点 x (即 mid), 即 $x = \lfloor (low + high) / 2 \rfloor$ 。把

$[low, high]$ 分为三部分： $[low, x-1]$ 、 x 、 $[x+1, high]$

先用给定值 key 与 mid 指定的值相比，若相等，则表示查找成功。若不相等，此时存在如下两种情况：

(1) key 值小于 mid 所指的值，说明 key 在 low 和 mid 之间，则令指针 $high = mid - 1$ ，在表的前半部分再取中间位置的记录进行比较；

(2) key 值大于 mid 所指的值，说明 key 在 mid 和 $high$ 之间，则令指针 $low = mid + 1$ ，在表的后半部分再取中间位置的记录进行比较。

如此反复进行，直至找到或查完全表而查不到为止。

记录编号	1	2	3	4	5	6	7	8	9	10	11
关键字值	5	13	19	21	37	56	64	75	80	88	92
	$\uparrow low$					$\uparrow mid$			$\uparrow high$		
	$low=1$					$mid=6$			$high=11$		

(a) 初始时

$\uparrow low$	$\uparrow mid$	$\uparrow high$				$\uparrow low$	$\uparrow mid$	$\uparrow high$
$low=1$	$mid=3$	$high=5$				$low=7$	$mid=9$	$high=11$
		$\uparrow low$					$\uparrow low$	
		$low=4$					$low=10$	
	$\uparrow mid$						$\uparrow mid$	
	$mid=4$						$mid=10$	

(b) $key=21$ 时查找成功

(c) $key=85$ 时查找不成功

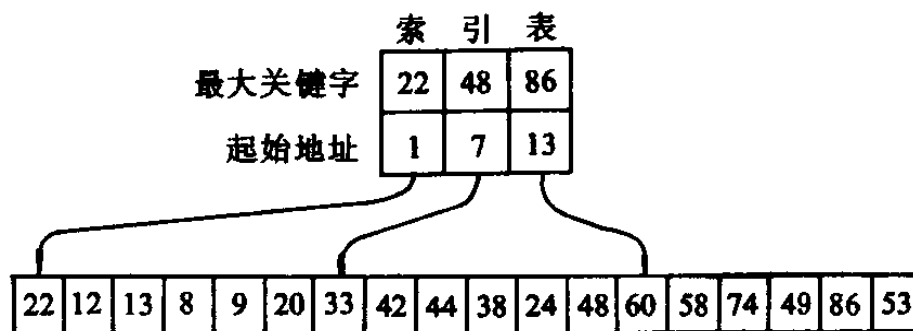
一个有序表

(1) 初始时， $low=1, high=11, mid=\lfloor (1+11) / 2 \rfloor = 6$

(2) 当 $key=21$ 时，再令 $mid=\lfloor (1+5) / 2 \rfloor = 3$ ， $r[mid].key=19$ ，则令 $low=3+1=4$ ，再令 $mid=\lfloor (4+5) / 2 \rfloor = 4$ ， $r[mid].key=21$

(3) 当 $key=85$ 时，大于 $r[mid].key=56$ ，则令 $low=6+1=7$ ，再令 $mid=\lfloor (7+11) / 2 \rfloor = 9$ ， $r[mid].key=80$ ，则令 $low=9+1=10$ ，再令 $mid=\lfloor (10+11) / 2 \rfloor = 10$ ， $r[mid].key=88$ ，令 $high=mid-1=10-1=9 < low$

3. 分块查找



线性表及其索引表

$(R_1, R_2, \dots, R_6)$

$(R_7, R_8, \dots, R_{12})$

(R13, R14, ..., R18)

7.3 动态查找

动态查找的特点是：表结构本身是在查找过程中动态生成。

1. 二叉排序树

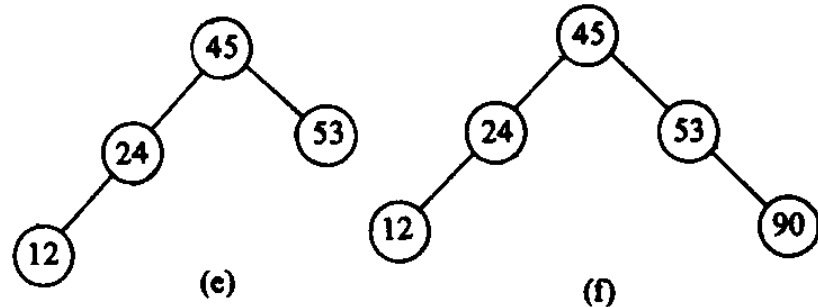
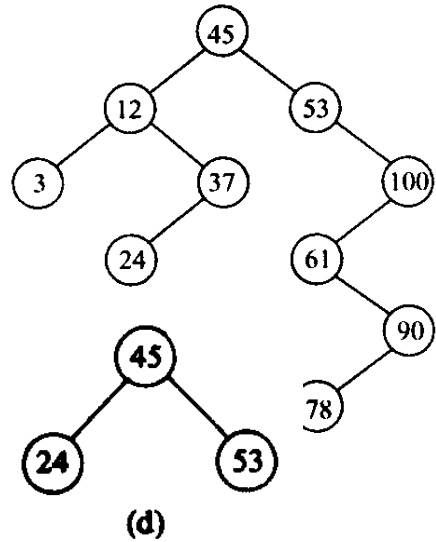
二叉排序树 T 是一棵二叉树，它或者是空，或者具有下列 3 个性质：

- (1) 如果 T 的根结点的左子树非空，它的左子树所有结点的关键字均小于 T 的根结点的关键字；
- (2) 如果 T 的根结点的右子树非空，它的右子树所有结点的关键字均大于 T 的根结点的关键字；
- (3) T 的根结点的左子树和右子树也均是二叉排序树。

如果按照中序遍历一棵二叉查找树所得到的结点的序列是一个递增序列。

3, 12, 24, 37, 45, 53, 61, 78, 90, 100

{45, 24, 53, 45, 12, 24, 90}



二叉排序树的构造过程

(a) 空树；(b) 插入 45；(c) 插入 24；(d) 插入 53；(e) 插入 12；(f) 插入 90。

第八章 内部排序

8.1 排序的有关概念

排序是将一个数据元素(或记录)的任意序列，重新排列成一按关键字(或排序码)有序的序列。

若对于关键字相同的两条记录 R_i 和 R_j ，经排序后仍保持同样的相对顺序，则该排序方法是稳定的，否则是不稳定的。

内部排序：指待排序记录存放在计算机内存中进行的排序。

外部排序：指待排序记录的数量很大，排序过程中需对外存进行访问。

一. 直接插入排序

直接插入排序(straight insertion sort)一种最简单的排序方法。

- (1) 算法思想：逐个处理待排序列中的记录，将其与前面已经排好序的子序中的记录进行比较，确

定要插入的位置，并将记录插入到子序中。

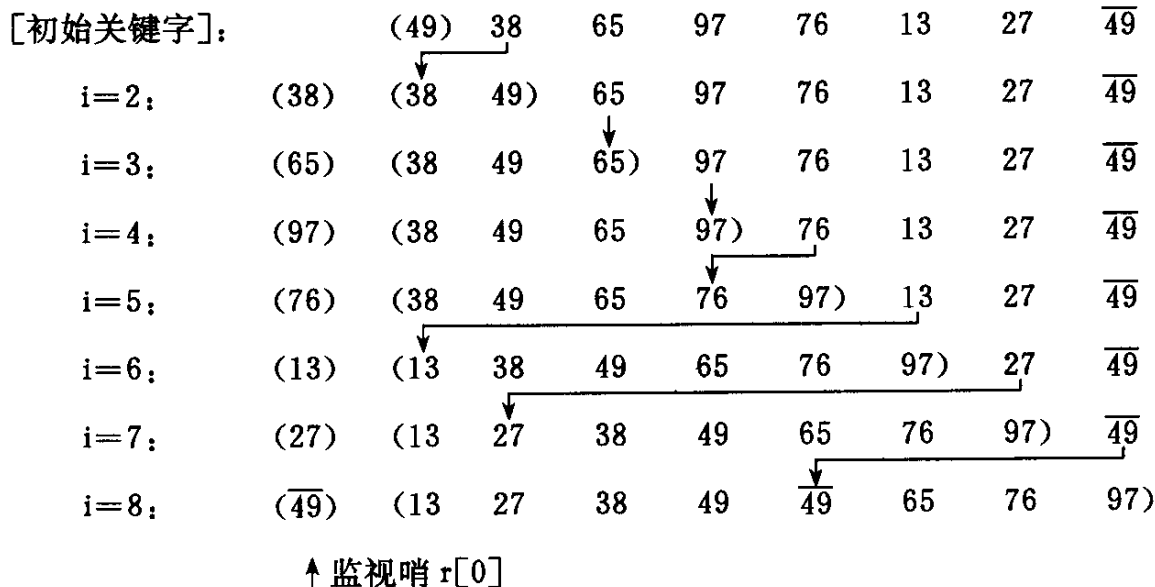
①开始时，把第①个记录看成是已经排好序的子序，这时子序中只有一个记录；

②从第②个记录起到最后一个记录，依次将记录和前面子序中的记录比较，确定记录插入的位置；

③将记录插入到子序中，子序记录个数加1，直至子序长度和原来待排序列长度一致时结束。

直接插入排序示例

二. 折半插入排序



例：28, 13, 72, 85, 39, 41, 6, 20

(1) [6 13 28 39 41 72 85] 20

l=1 m=4 h=7

20 < 39, m-1=3 → h

(2) [6 13 28] 39 41 72 85 20

l=1 m=2 h=3

20 > 13, m+1=3 → l

(3) 6 13 [28] 39 41 72 85 20

l=m=h=3

20 < 28, m-1=2 → h

l > h, 折半结束, l=3 为插入位置

(4) 6 13 20 28 39 41 72 85

折半插入排序示例

三. 希尔排序

1. 算法思想

先将 n 个待排记录序列分割成若干个子序列，然后对各子序列分别进行排序，当整个序列中的记录“基本有序”时，再对全体记录进行一次直接插入排序。

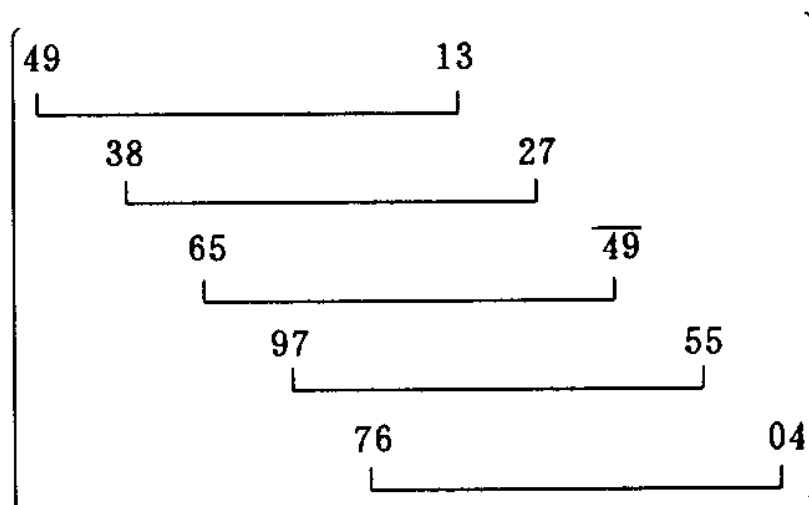
①取定一个正整数 $d_1 < n$ ，把全部记录按此间隔值，从第一个记录起进行分组，所有距离为 d_1 倍数的记录放一组中，在各组内进行直接插入排序；

②取定一个正整数 $d_2 < d_1$ ，重复上述分组和排序工作，直至取 $d_i = 1$ 为止。

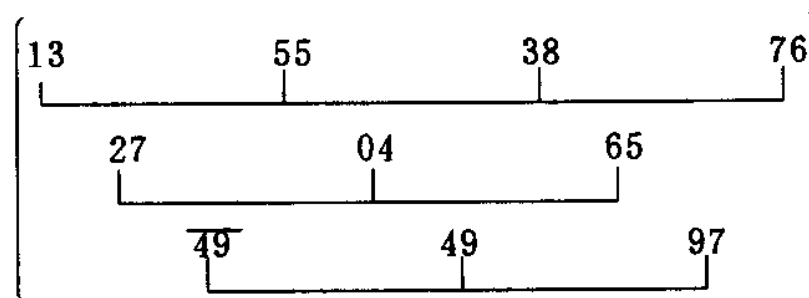
49 38 65 97 76 13 27 49 55 04

取 $d_1 = \lfloor n / 2 \rfloor = 4$, $d_{i+1} = \lfloor d_i / 2 \rfloor$, 排序过程如下：

〔初始关键字〕: 49 38 65 97 76 13 27 49 55 04



一趟排序结果: 13 27 49 55 04 49 38 65 97 76



二趟排序结果: 13 04 49 38 27 49 55 65 97 76

三趟排序结果: 04 13 27 38 49 49 55 65 76 97

希尔排序示例

希尔排序是不稳定的排序方法。

8.2 快速排序

交换排序法，其基本方法是：两两比较待排序记录的关键字，并交换不满足顺序要求的那些偶对，直到全部满足为止。

一. 起泡排序（冒泡排序）

起泡排序 (bubble sort) 是一种简单但是著名的交换排序方法。

(1) 算法思想。

每次进行相邻两个元素关键字的比较，如果不符合次序立即交换。

①将第1个记录的关键字 k_1 和第2个记录的关键字 k_2 进行比较，若为逆序（即 $k_1 > k_2$ ），则将两个记录进行交换，若为正序则保持原序；

②将第2个记录的关键字 k_2 和第3个记录的关键字 k_3 进行比较，重复上述排序过程，直到第 $n-1$ 个记录和第 n 个记录的关键字进行比较、交换为止；

③这样从 (k_1, k_2) 到 (k_{n-1}, k_n) 的 $n-1$ 次比较和交换（即上述①和②）的排序过程称做第一趟起泡排序

④进行第2趟起泡排序，对前 $n-1$ 个记录进行同样的操作，其结果是使得关键字次大的记录被安置到第 $n-1$ 个记录的位置上；

依此类推，第 i 趟冒泡排序是从第1个记录到第 $n-i+1$ 个记录之间依次比较和交换。重复以上过程直到没有记录需要交换为止。

49	38	65	97	76	13	27	49	—		
49	38	38	38	38	13	13				
38	49	49	49	13	27	27				
65	65	65	13	27	38	38				
97	76	13	27	49	49					
76	13	27	49	49						
13	27	49	65							
27	49	76								
49	97									
初始关键字	第一趟排序后	第二趟排序后	第三趟排序后	第四趟排序后	第五趟排序后	第六趟排序后				

起泡排序的过程示意图。

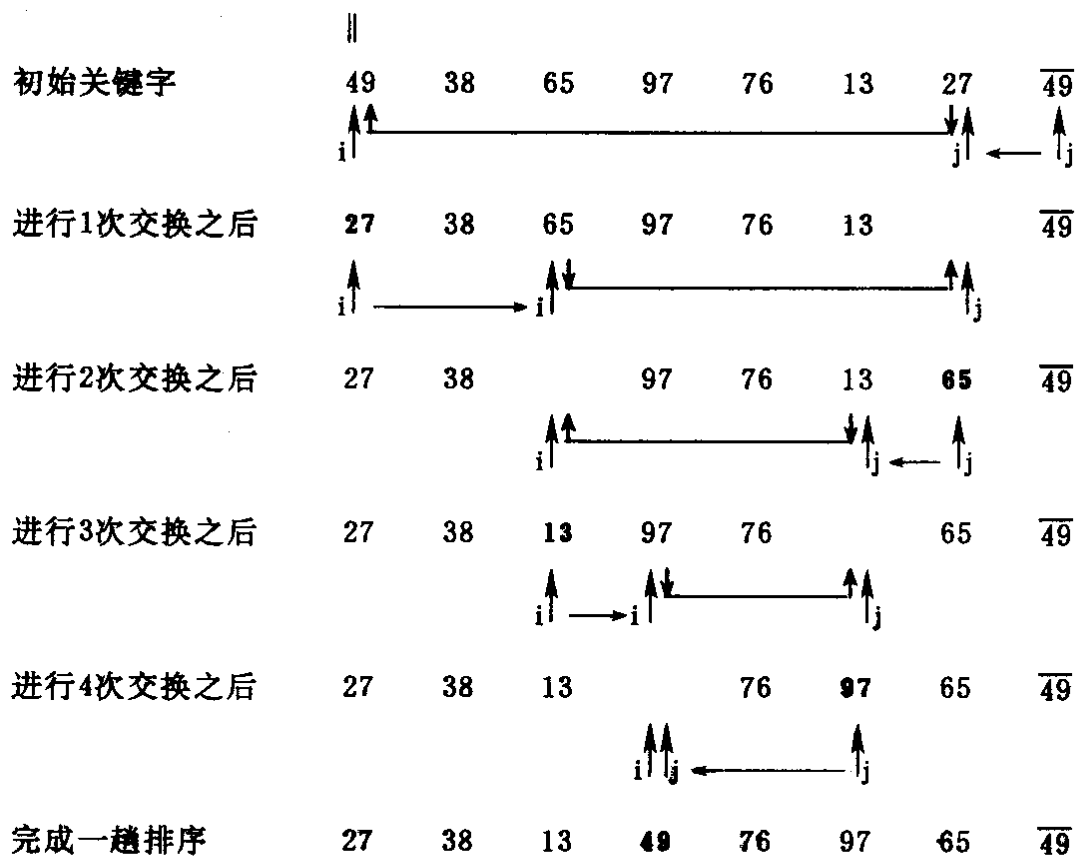
起泡排序是稳定的。

二. 快速排序

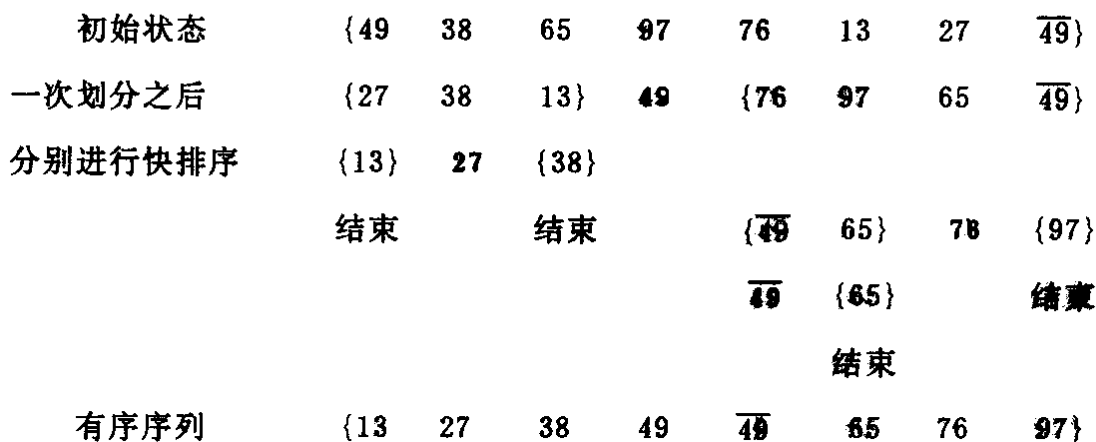
快速(quick sort)，又称分区交换排序法。

1. 算法思想。每一步都要把待排序的表里的某个元素放到它在表中的最终位置，同时在这个元素的前面和后面各形成一个子表，在前子表中的所有元素的关键字都比这个元素的关键字小，而在后子表中的都比它大，则可以分别对这两部分记录继续进行排序，直到最后每个子表都只有一个元素，以达到整个序列有序。

49 38 65 97 76 13 27 49



(a)



快速排序的过程示意图

快速排序是不稳定的。

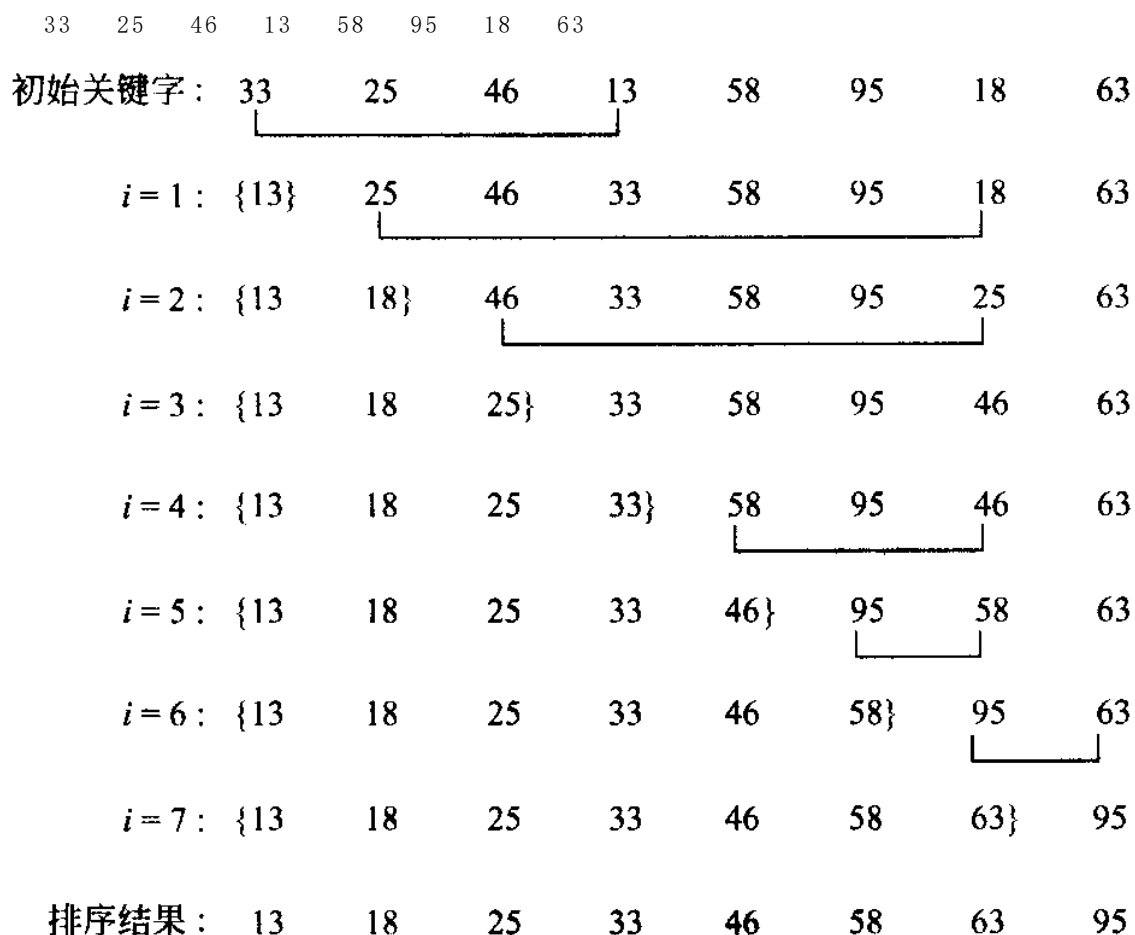
8.3 选择排序法

选择排序法的基本思想是：每一趟在 $n-i+1$ ($i=1, 2, \dots, n-1$) 个记录中选取关键字最小的记录，顺序放在已排序的记录序列的最后，作为有序序列中第 i 个记录，直到全部排序完成。

一、简单选择排序（直接选择排序）

简单选择排序的基本思想是：设 n 个待排序的记录存放在 $L.r[1..n]$ 中，对 n 个待排序记录进行 $n-1$ 趟扫描：

- ①第一趟扫描选出 n 个记录中关键字值最小的记录，并与 $L.r[1]$ 记录交换位置；
 - ②第二趟扫描选出余下的 $n-1$ 个记录中关键字值次最小的记录，并与 $L.r[2]$ 记录交换位置；
- 依此类推，直至第 $n-1$ 趟扫描结束，所有记录有序为止。



简单选择排序的过程示意图。

直接选择排序是不稳定的。

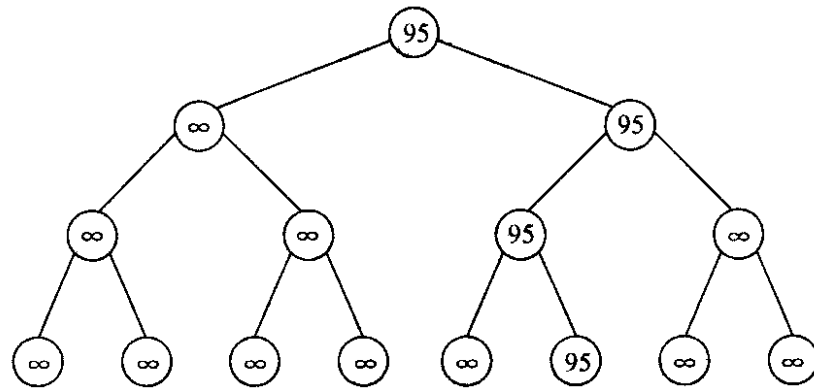
二. 树型选择排序

树型选择排序的基本思想是：

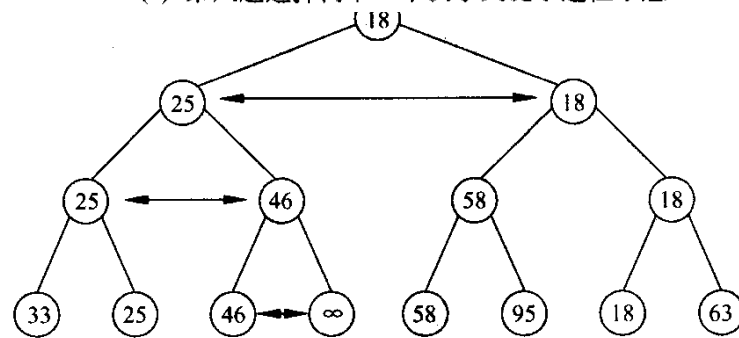
(1) 把 n 个记录的关键字值两两比较，将其中关键字值较小的 $n/2$ 个记录取出来作为第一步比较的结果保存下来，然后将这 $n/2$ 个关键字进行两两比较，继续选择具有较小关键字值的记录，直至选出一个最小关键字值；

(2) 对余下的 $n-1$ 个记录的关键字值进行第二趟两两比较，再选出一个次小的关键字；如此反复，直至排序结束为止。

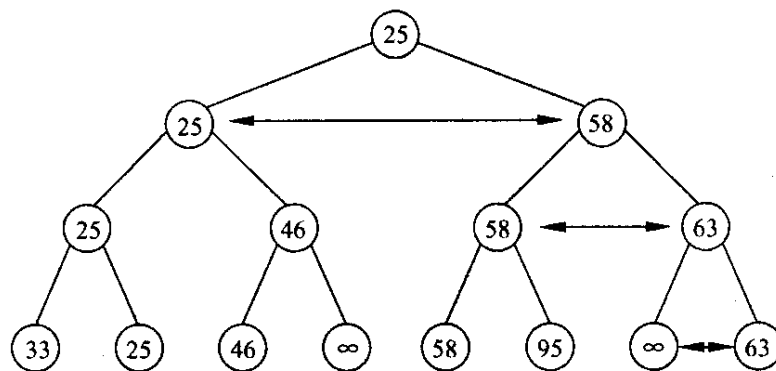
33 25 46 13 58 95 18 63



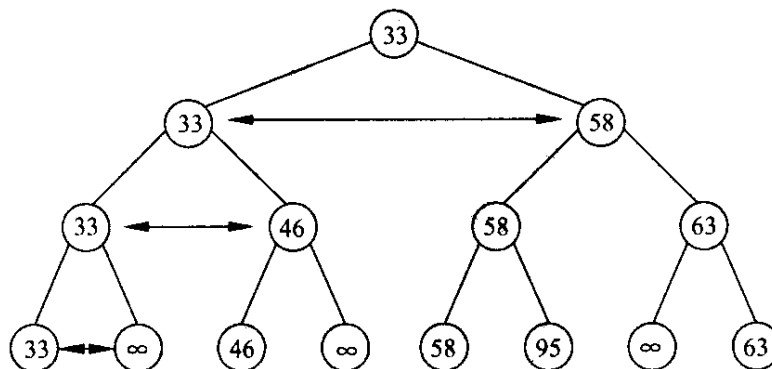
(e) 第八趟选择再下一个次小关键字过程示意



(b) 第二趟选择次小关键字过程示意



(c) 第三趟选择下一个次小关键字过程示意



(d) 第四趟选择再下一个次小关键字过程示意

树型选择排序过程的示例

三. 堆排序

堆排序的基本思想是：对一组待排序的关键字，首先把它们按堆的定义排成一个序列（称为建堆），这就找到了最小的关键字，然后将最小的关键字取出，用剩下的关键字再建堆，便得到次最小的关键字，如此反复执行，直到将全部关键字排好序为止。

堆排序是不稳定的。

8.4 归并排序法(merging sort)

归并排序是又一类不同的排序方法。归并排序则要求待排序文件已经部分有序。“归并”的含义是将两个或两个以上的有序表组合成一个新的有序表。

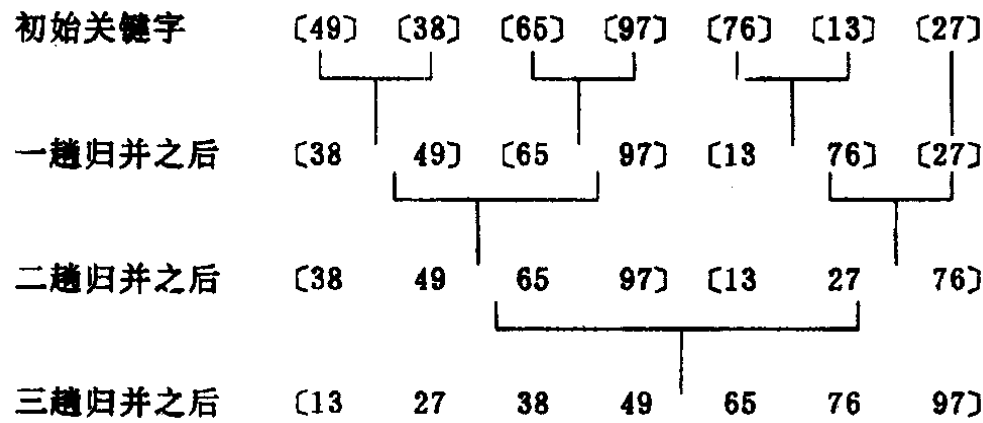
1. 算法思想

将两个或两个以上的有序序列归并成一个有序序列，以得到一个完全排序的文件。合并时只要比较各子文件的第一个记录的关键字，最小的一个就是排序后第一个记录的关键字，取出这个记录，继续比较各子文件的第一个记录，便找出排序后第二个记录，如此继续下去，只要经过一遍扫描，就可以得到排序结果。

例如，设有两个部分排序的子文件 F1、F2，对应的关键字集合分别是：F1：{23, 71, 72, 73} F2：{5, 16, 68, 94}

经过一遍合并便得到排序的文件 F：{5, 16, 23, 68, 71, 72, 73, 94}

2-路归并排序的过程如下图所示。



2-路归并排序的过程示例

归并排序与快速排序和堆排序相比，最大特点在于它是一种稳定的排序方法。

8.5 基数排序法

8.6 各种内部排序法性能比较

排序方法	平均时间	最坏情况	辅助存储
简单排序	$O(n^2)$	$O(n^2)$	$O(1)$
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$
堆排序	$O(n \log n)$	$O(n \log n)$	$O(1)$
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$
基数排序	$O(d(n + rd))$	$O(d(n + rd))$	$O(rd)$

- (1) 比较两个关键字的大小。
- (2) 将记录从一个位置移动到另一个位置。

1. 从依据的不同原则看, 内部排序大致可分为: 插入排序、交换排序、选择排序、归并排序和记数排序。

2. 从时间复杂度看, 内部排序可分为:

(1) 简单排序方法(包括直接插入排序、冒泡排序、选择排序)

其时间复杂度为 $O(n^2)$;

(2) 先进的排序方法(包括堆排序、归并排序、快速排序)

其时间复杂度为 $O(n \log_2 n)$;

(3) 基数排序, 其时间复杂度为 $O(d \cdot n)$ 。

3. 从空间复杂度看, 内部排序可分为:

(1) 归并排序, 其空间复杂度为 $O(n)$;

(2) 快速排序, 其空间复杂度为 $O(\log_2 n)$;

(3) 其他排序方法, 其空间复杂度为 $O(1)$ 。

4. 从稳定性看, 内部排序可分为:

(1) 稳定的排序方法(包括直接插入排序, 冒泡排序, 归并排序);

(2) 不稳定的排序方法(希尔排序、选择排序、快速排序、堆排序)。{4, 3, 4, 2}

(1) 直接插入排序适合于原始数据基本有序或 n 较小 ($n \leq 16$) 的情况。

(2) 快速排序适合于原始数据杂乱无章, n 较大且对稳定性没有要求的情况。

(3) 堆排序(或归并排序)适合于 n 较大, 原始数据可能出现正序或逆序且对稳定性没有要求的情况。

(4) 归并排序适合于 n 较大, 内存空间允许且要求排序稳定的情况。

(5) 基数排序的效率与原始数据的排列顺序无关, 直接选择排序的比较次数与初始状态无关, 但记录移动次数与初始状态有关。

在最好情况下, 直接插入排序和冒泡排序最快; 在平均情况下, 快速排序最快; 在最坏情况下, 堆排序和归并排序最快。

在以比较为基础的排序方法中, 最少比较次数为 $n-1$, 最多比较次数为 $n(n-1)/2$, 最坏情况下能达到的最好的时间复杂度为 $O(n \log_2 n)$ 。